Full Mass Matrix MPM or FMPM

John A. Nairn

March 25, 2026

Contents

1 MPM Extrapolations **5**
- 1.1 Grid To Particle Extrapolations . . . 5
- 1.2 Particle to Grid Extrapolations . . . 5
- 1.3 Full Mass Matrix Expansion . . . 6
- 1.4 Velocity Extrapolation . . . 7
  - 1.4.1 Some Explicit Summation Results . . . 8
  - 1.4.2 Another Comparison to XPIC . . . 8
  - 1.4.3 Revised Velocity Extrapolation . . . 9
  - 1.4.4 Estimating the Inverse Error . . . 10
- 1.5 Another Possible Change for BC and Contact . . . 10
  - 1.5.1 Prior notes for BC and Contact . . . 11
- 1.6 Components of Approximate Full Mass Matrix Inverse . . . 13
- 1.7 Some Extrapolation Results . . . 13

2 The MPM Algorithm - Single Material Mode **15**
- 2.1 MPM Time Step Tasks . . . 15
  - 2.1.1 Extrapolate Mass and Momentum to the Grid . . . 15
  - 2.1.2 Update Strains First . . . 16
  - 2.1.3 Extrapolate Forces . . . 17
  - 2.1.4 Update Grid Momenta . . . 17
  - 2.1.5 Update Particles . . . 17
  - 2.1.6 Update Strains Last . . . 18
  - 2.1.7 Move Cracks Tasks . . . 18
- 2.2 Bare Bones MPM Algorithm . . . 18
- 2.3 Particle Update Details . . . 19
  - 2.3.1 FMPM Updates . . . 19
  - 2.3.2 Prior FLIP MPM Updates . . . 19
  - 2.3.3 Prior XPIC Updates . . . 20
  - 2.3.4 Various PIC Updates . . . 21
  - 2.3.5 Particle Update Code Comparison . . . 22
- 2.4 Velocity Boundary Conditions - Background . . . 22
- 2.5 Velocity Boundary Conditions - Revised . . . 25

3 FMPM With Contact **29**
- 3.1 Full Mass Matrix Equations . . . 29
  - 3.1.1 FMPM(k) Contact Options . . . 30
  - 3.1.2 Comparison of FMPM(k) Methods . . . 31

Chapter 1

MPM Extrapolations

1.1 Grid To Particle Extrapolations

Let lower case denote grid (or Eulerian frame) quantities and upper case denote particle (or Lagrangian frame) quantities. We define S to be a matrix of grid-based shape functions to extrapolate from nodes to any position (normally to a particle position). Thus:

$$\boldsymbol{Q}_p = \sum_i S_{pi}\boldsymbol{q}_i \qquad \text{and} \qquad 1 = \sum_i S_{pi} \qquad \text{and} \qquad \boldsymbol{Q} = \mathsf{S}\boldsymbol{q} \tag{1.1}$$

The first extrapolates any nodal quantity, q_i, to a particle quantity, Q_p, and S_{pi} is shape function for node i at position of particle p. The second requires partition of unity for the shape functions. The third writes it is as a linear transformation where $\boldsymbol{Q}$ is vector of particle properties, $\boldsymbol{q}$ is vector of nodal quantities, and S is the $N \times n$ transformation matrix (N particles and n nodes).

1.2 Particle to Grid Extrapolations

Extrapolating properties from particle to grid would like inverse of S. Becase S is not square, we need to use a pseudo inverse denoted here by S^+ which is $n \times N$ and was first derived in original Sulsky MPM paper using weighted least squares. Imagine some grid velocity give by $\boldsymbol{v}$ and then extrapolate them to the particles using $\tilde{\boldsymbol{V}} = \mathsf{S}\boldsymbol{v}$. We find S^+ such that $\boldsymbol{v} = \mathsf{S}^+\boldsymbol{V}$ minimizes squared errors between $\boldsymbol{V}$ and $\tilde{\boldsymbol{V}}$. A mass-weighted error minimization is:

$$\Omega = \sum_p M_p\big(\boldsymbol{V}_p - \tilde{\boldsymbol{V}}_p\big)^2 = \sum_p M_p\big(\boldsymbol{V}_p - \sum_i S_{pi}\boldsymbol{v}_i\big)^2 \tag{1.2}$$

$$0 = \frac{d\Omega}{d\boldsymbol{v}_j} = -2\left(\sum_p S_{pj}M_p\boldsymbol{V}_p - \sum_i\sum_p M_pS_{pj}S_{pi}\boldsymbol{v}_i\right) \tag{1.3}$$

$$\sum_i \tilde{m}_{ji}\boldsymbol{v}_i = \sum_p S^T_{jp}M_p\boldsymbol{V}_p \tag{1.4}$$

where $\tilde{m}_{ji} = \sum_p M_pS^T_{jp}S_{pi}$ is an element of the full (and symmetric) $n \times n$ mass matrix $\tilde{\mathsf{m}}$. In matrix form:

$$\tilde{\mathsf{m}}\boldsymbol{v} = \mathsf{S}^T\boldsymbol{P}, \qquad \boldsymbol{v} = \tilde{\mathsf{m}}^{-1}\mathsf{S}^T\boldsymbol{P} = \tilde{\mathsf{S}}^+\boldsymbol{V}, \qquad \text{with} \qquad \tilde{\mathsf{S}}^+ = \tilde{\mathsf{m}}^{-1}\mathsf{S}^T\mathsf{M} \tag{1.5}$$

where $\boldsymbol{P}$ is a vector of particle momenta and $\mathsf{M} = \text{diag}(\boldsymbol{M})$ is $N \times N$ diagonal matrix formed from vector $\boldsymbol{M}$ of particle masses (when using same letter, serif form is a vector and sans-serif is a matrix). The $\tilde{}$ on $\tilde{\mathsf{m}}$ and $\tilde{\mathsf{S}}^+$ indicate terms using the full mass matrix.

In standard MPM, $\tilde{\mathsf{m}}$ is replaced m, which is a diagonal lumped mass matrix, with i^{th} diagonal element defined by:

$$m_i = \sum_j \tilde{m}_{ji} = \sum_p S_{pi} M_p = \sum_p S^T_{ip} M_p \quad \text{or} \quad \boldsymbol{m} = \mathsf{S}^T \boldsymbol{M} \quad \text{and} \quad \mathsf{m} = \text{diag}(\boldsymbol{m}) \tag{1.6}$$

where $\boldsymbol{m}$ is vector of nodal masses. The pseudo inverse matrix for a lumped mass matrix (without˜) is:

$$\mathsf{S}^+ = \mathsf{m}^{-1}\mathsf{S}^T\mathsf{M} \quad \Longrightarrow \quad S^+_{ip} = \sum_k \sum_l \frac{\delta_{ik}}{m_i} S^T_{kl} \delta_{lp} M_p = \frac{S_{pi} M_p}{m_i} \tag{1.7}$$

1.3 Full Mass Matrix Expansion

To write full matrix matrix in new form, we note that the $(jp)^{th}$ element of $\mathsf{S}^T\mathsf{M}$ is:

$$(\mathsf{S}^T\mathsf{M})_{jp} = \sum_q S^T_{jq} \mathsf{M}_{qp} = \sum_q S^T_{jq} M_p \delta_{qp} = M_p S^T_{jp} \tag{1.8}$$

The full mass matrix can then be written as:

$$(\mathsf{S}^T\mathsf{M}\mathsf{S})_{ji} = \sum_p (\mathsf{S}^T\mathsf{M})_{jp} S_{pi} = \tilde{m}_{ji} \qquad \Longrightarrow \qquad \tilde{\mathsf{m}} = \mathsf{S}^T\mathsf{M}\mathsf{S} \tag{1.9}$$

Using S^+ with lumped mass matrix, the full mass matrix is

$$\tilde{\mathsf{m}} = \mathsf{m}\mathsf{S}^+\mathsf{S} = \mathsf{m}\big(\mathsf{I}_n - (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})\big) = \mathsf{m}\big(\mathsf{I}_n - \mathsf{A}\big) \tag{1.10}$$

where I_n is an $n \times n$ identity matrix and

$$\mathsf{A} = \mathsf{I}_n - \mathsf{S}^+\mathsf{S} \tag{1.11}$$

Note that $\mathsf{S}^+\mathsf{S}$ is $n \times n$ and symmetric. The full mass matrix inverse is then

$$\tilde{\mathsf{m}}^{-1} = (\mathsf{I}_n - \mathsf{A})^{-1}\mathsf{m}^{-1} = (\mathsf{I}_n + \mathsf{A} + \mathsf{A}^2 + \mathsf{A}^3 + \cdots)\mathsf{m}^{-1} \tag{1.12}$$

Using one term is lumped matrix inverse and additional terms approximate the full mass matrix better.

The psuedo-inverse shape function matrix using full mass matrix expanded with k terms is:

$$\tilde{\mathsf{S}}^+_k = \big(\mathsf{I}_n + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S}) + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^2 + \cdots + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^{k-1}\big)\mathsf{S}^+ \tag{1.13}$$

Increasing order approximations to $\tilde{\mathsf{S}}^+$ would be

$$\tilde{\mathsf{S}}^+_1 = \mathsf{S}^+ \tag{1.14}$$

$$\tilde{\mathsf{S}}^+_2 = \big(2\mathsf{I}_n - \mathsf{S}^+\mathsf{S}\big)\mathsf{S}^+ \tag{1.15}$$

$$\vdots \tag{1.16}$$

$$\tilde{\mathsf{S}}^+_k = \left(\mathsf{I}_n + \sum_{j=1}^{k-1}\sum_{\ell=0}^{j}(-1)^\ell \binom{j}{\ell}\big(\mathsf{S}^+\mathsf{S}\big)^\ell\right)\mathsf{S}^+ \tag{1.17}$$

For more efficient implementation, we rearrange:

$$\tilde{\mathsf{S}}_k^+ = \left(k\mathsf{I}_n + \sum_{\ell=1}^{k-1}\sum_{j=\ell}^{k-1}(-1)^\ell\binom{j}{\ell}\left(\mathsf{S}^+\mathsf{S}\right)^\ell\right)\mathsf{S}^+ \tag{1.18}$$

$$= \left(k\mathsf{I}_n + \sum_{\ell=2}^{k}\sum_{j=\ell}^{k}(-1)^{\ell+1}\binom{j-1}{\ell-1}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{S}^+ \tag{1.19}$$

$$= \left(k\mathsf{I}_n + \sum_{\ell=2}^{k}C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{S}^+ = \left(\sum_{\ell=1}^{k}C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{S}^+ \tag{1.20}$$

where

$$C_{k\ell} = (-1)^{\ell+1}\sum_{j=\ell}^{k}\binom{j-1}{\ell-1} \qquad \text{and} \qquad C_{k1} = k \tag{1.21}$$

1.4 Velocity Extrapolation

Velocity extrapolation with approximate full mass matrix becomes:

$$\boldsymbol{v} = \tilde{\mathsf{S}}_k^+\boldsymbol{V} = \left(\sum_{\ell=1}^{k}C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{m}^{-1}\mathsf{S}^T\mathsf{M}\boldsymbol{V} = \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p} \tag{1.22}$$

where

$$\tilde{\mathsf{m}}_k^{-1} = \left(\sum_{\ell=1}^{k}C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{m}^{-1} \tag{1.23}$$

We can simplify C_{kl} using the binomial theorem or

$$\binom{k}{\ell} = \binom{k-1}{\ell-1} + \binom{k-1}{\ell} \tag{1.24}$$

$$= \binom{k-1}{\ell-1} + \binom{k-2}{\ell-1} + \binom{k-2}{\ell} \tag{1.25}$$

$$= \binom{k-1}{\ell-1} + \binom{k-2}{\ell-1} + \binom{k-3}{\ell-1} + \binom{k-3}{\ell} \tag{1.26}$$

$$\vdots \tag{1.27}$$

$$= \binom{k-1}{\ell-1} + \binom{k-2}{\ell-1} + \binom{k-3}{\ell-1} + \cdots + \binom{\ell-1}{\ell-1} \tag{1.28}$$

$$= \sum_{j=\ell}^{k}\binom{j-1}{\ell-1} \tag{1.29}$$

In other words:

$$C_{kl} = (-1)^{\ell+1}\binom{k}{\ell} \tag{1.30}$$

We can also show by reverse induction. The relation holds for $\ell = k$. Now assume it holds for ℓ and prove it holds for $\ell - 1$:

$$\binom{k}{\ell-1} = \binom{k+1}{\ell} - \binom{k}{\ell} = \sum_{j=\ell}^{k+1}\binom{j-1}{\ell-1} - \sum_{j=\ell}^{k}\binom{j-1}{\ell-1} = \binom{k}{\ell-1} \quad \checkmark \tag{1.31}$$

Next compare to XPIC(k) (using k instead of m for XPIC order), projection onto non-null-space of velocities uses projector:

$$\mathsf{P}_k = \mathsf{I}_N - (\mathsf{I}_N - \mathsf{S}\mathsf{S}^+)^k \tag{1.32}$$

where I_N is an $N \times N$ identity matrix. The projection expands to:

$$\begin{aligned}
\mathsf{P}_k \boldsymbol{V} &= \sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} (\mathsf{S}\mathsf{S}^+)^\ell \boldsymbol{V} && (1.33)\\
&= \mathsf{S}\left(\sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} (\mathsf{S}^+\mathsf{S})^{\ell-1} \right) \mathsf{S}^+ \boldsymbol{V} && (1.34)\\
&= \mathsf{S}\left(\sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} (\mathsf{S}^+\mathsf{S})^{\ell-1} \right) \mathsf{m}^{-1} \mathsf{S}^T \mathsf{M} \boldsymbol{V} && (1.35)\\
&= \mathsf{S}\tilde{\mathsf{m}}_k^{-1} \boldsymbol{p} = \mathsf{S}\boldsymbol{v}(k) && (1.36)
\end{aligned}$$

In other words, $\boldsymbol{v}(k)$ used in FMPM implementation (and also defined in `Extrap.pdf` notes) is identical to $\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}$. XPIC($k$) should be interpreted as finding grid velocities using an approximate full mass matrix. By using XPIC(k) methods other places in the MPM time step, the entire method can be cast as an approximate full mass matrix method.

1.4.1 Some Explicit Summation Results

As an explicit sum, the full mass matrix inverse is:

$$\tilde{\mathsf{m}}_k^{-1} = \left(\sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} \left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1} \right) \mathsf{m}^{-1} \tag{1.37}$$

A combination that appears in some calculations includes

$$\tilde{\mathsf{m}}_k^{-1}\mathsf{m} - \mathsf{I}_n = \mathsf{A}(\mathsf{I}_n + \mathsf{A} + \mathsf{A}^2 + \cdots + \mathsf{A}^{k-2}) = (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})\tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{m} \tag{1.38}$$

For $k = 1$ this results is zero. The XPIC projection operator is

$$\mathsf{P}_k = \mathsf{S}\left(\sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} (\mathsf{S}^+\mathsf{S})^{\ell-1} \right) \mathsf{S}^+ = \sum_{\ell=1}^{k} (-1)^{\ell+1} \binom{k}{\ell} (\mathsf{S}\mathsf{S}^+)^\ell = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\mathsf{m}\mathsf{S}^+ \tag{1.39}$$

As a geometric sum, we could try

$$\tilde{\mathsf{m}}_k^{-1} = (\mathsf{I}_n - \mathsf{A})^{-1}(\mathsf{I}_n - \mathsf{A}^k)$$

Then

$$\tilde{\mathsf{m}}_k = (\mathsf{I}_n - \mathsf{A}^k)^{-1}(\mathsf{I}_n - \mathsf{A})$$

1.4.2 Another Comparison to XPIC

Using geometric series, we can show that

$$(\mathsf{I} + \mathsf{A} + \mathsf{A}^2 + \mathsf{A}^3 + \cdots + \mathsf{A}^{k-1})\big(\mathsf{I} - \mathsf{A}\big) = \mathsf{I} - \mathsf{A}^k$$

where A is any matrix. We apply to $\mathsf{P}_k = \mathsf{I}_N - (\mathsf{I}_N - \mathsf{S}\mathsf{S}^+)^k$ with $\mathsf{A} = \mathsf{I}_N - \mathsf{S}\mathsf{S}^+$ to derive

$$\begin{aligned}
\mathsf{P}_k &= \big(\mathsf{I}_N + (\mathsf{I}_N - \mathsf{S}\mathsf{S}^+) + (\mathsf{I}_N - \mathsf{S}\mathsf{S}^+)^2 + \cdots + (\mathsf{I}_N - \mathsf{S}\mathsf{S}^+)^{k-1}\big)\mathsf{S}\mathsf{S}^+ && (1.40)\\
&= \mathsf{S}\mathsf{I}_n\mathsf{S}^+ + \mathsf{S}(\mathsf{I}_n - \mathsf{S}^+\mathsf{S})\mathsf{S}^+ + \mathsf{S}(\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^2\mathsf{S}^+ + \cdots + \mathsf{S}(\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^{k-1}\mathsf{S}^+ && (1.41)\\
&= \mathsf{S}\big(\mathsf{I}_n + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S}) + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^2 + \cdots + (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^{k-1}\big)\mathsf{S}^+ && (1.42)\\
&= \mathsf{S}\tilde{\mathsf{S}}_k^+ && (1.43)
\end{aligned}$$

Then

$$\mathsf{P}_k = \mathsf{S}\tilde{\mathsf{S}}_k^+ = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\mathsf{S}^T\mathsf{M} = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\mathsf{m}\mathsf{S}^+ \tag{1.44}$$

1.4.3 Revised Velocity Extrapolation

Rather than proceed through derivations above, simply retain the initial form and write

$$\boldsymbol{v}(k) = \sum_{\ell=1}^{k} \Delta\boldsymbol{v}(\ell) \quad \text{where} \quad \Delta\boldsymbol{v}(\ell) = \boldsymbol{v}(\ell) - \boldsymbol{v}(\ell-1) = \mathsf{A}^{\ell-1}\boldsymbol{v}(1) = \mathsf{A}\Delta\boldsymbol{v}(\ell-1)$$

where $\boldsymbol{v}(1)$ is lumped mass velocity on the grid (initial or updated velocity). This recursion relation can be found be simple changes to one derived in XPIC paper (and used in first FMPM paper). In fact it is simpler (no need for scaling term or sign changes). The following pseudo-code outlines a full mass matrix algorithm whose output is a calculation of $\boldsymbol{v}(k)$):

for each active node on the grid
  let $\boldsymbol{v}(k) = \boldsymbol{v}_{prev} \leftarrow \mathsf{m}^{-1}\boldsymbol{p}$
for each ℓ **in** 2 **to** k **do**
  let $\boldsymbol{v}_{next} \leftarrow 0$
  for each particle p **in** N **do**
    find ϑ_p **nodes with non-zero shape functions** (S_{pi})
    for each node i **in** ϑ_p **do**
      for each node j **in** ϑ_p **do**
        let $w \leftarrow S_{pi}S_{pj}$
        let $(\boldsymbol{v}_{next})_i \leftarrow (\boldsymbol{v}_{next})_i + \frac{M_p w}{m_i}(\boldsymbol{v}_{prev})_j$
  let $\boldsymbol{v}_{prev} \leftarrow \boldsymbol{v}_{prev} - \boldsymbol{v}_{next}$
  set controlled grid velocities in $\boldsymbol{v}_{prev}$ **to zero (1)**
  let $\boldsymbol{v}(k) \leftarrow \boldsymbol{v}(k) + \boldsymbol{v}_{prev}$
  impose crack contact in $v(k)$ **(and add changes to** $\boldsymbol{v}_{prev}$ **as well) (2)**

Note that $\boldsymbol{v}_{prev}$ and the end of each loop is $\Delta v(\ell)$ and is the change in velocities on increasing FMPM order by one. One could potentially use its magnitude to look for FMPM order convergence.

This method also resolves grid velocity BCs (see step (1)) and multimaterial contact (see step (2)). In brief, these constraints are applied to each increment in velocities. The end result will be correct velocities on controlled nodes and improved multimaterial contact (and a method that reverts to single material mode if using perfect contact material model).

1.4.4 Estimating the Inverse Error

The error in $\tilde{\mathsf{m}}_k^{-1}$ is then found (using geometric series) by evaluating

$$\begin{aligned}\tilde{\mathsf{m}}_k^{-1}\tilde{\mathsf{m}} &= (\mathsf{I}_n + \mathsf{A} + \mathsf{A}^2 + \mathsf{A}^3 + \cdots + \mathsf{A}^{k-1})\big(\mathsf{I}_n - \mathsf{A}\big) &(1.45)\\ &= \mathsf{I}_n - \mathsf{A}^k = \mathsf{I}_n - (\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^k &(1.46)\end{aligned}$$

The exact results would be I_n or the error is $(\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^k$.

1.5 Another Possible Change for BC and Contact

We can try to define the mass matrix to include velocity changes caused by boundary conditions or contact. The least square analysis changes to a mass-weighted scalar metric:

$$\begin{aligned}\Omega &= \sum_p M_p\big(\boldsymbol{V}_p - \tilde{\boldsymbol{V}}_p\big)\cdot\big(\boldsymbol{V}_p - \tilde{\boldsymbol{V}}_p\big) = \sum_p M_p\big(\boldsymbol{V}_p - \sum_i S_{pi}\boldsymbol{v}_i^*\big)\cdot\big(\boldsymbol{V}_p - \sum_i S_{pi}\boldsymbol{v}_i^*\big)\\ &= \sum_p M_p\left(\boldsymbol{V}_p\cdot\boldsymbol{V}_p - 2\sum_i S_{pi}\boldsymbol{V}_p\cdot\boldsymbol{v}_i^* + \sum_k\sum_i S_{pk}S_{pi}\boldsymbol{v}_k^*\cdot\boldsymbol{v}_i^*\right)\end{aligned}$$

Here $\boldsymbol{v}_i^* = \boldsymbol{v}_i + \Delta\boldsymbol{v}_i$ where $\boldsymbol{v}_i$ are velocities in absence of contact and boundary conditions and $\Delta\boldsymbol{v}_i$ is change in velocity for node i caused by contact and boundary conditions ($\Delta\boldsymbol{v}_i = 0$ for nodes remote from contact and boundary conditions). If a node has a grid velocity boundary condition, we can write

$$\Delta\boldsymbol{v}_i = (v_i^{(b)} - \boldsymbol{v}_i\cdot\hat{\boldsymbol{n}}_i^{(b)})\hat{\boldsymbol{n}}_i^{(b)} = v_i^{(b)}\hat{\boldsymbol{n}}_i^{(b)} - \hat{\boldsymbol{n}}_i^{(b)}\otimes\hat{\boldsymbol{n}}_i^{(b)}\boldsymbol{v}_i$$

This change sets velocity on node i in the $\hat{\boldsymbol{n}}_i^{(b)}$ direction to scalar $v_i^{(b)}$. The first form subtracts component of $\boldsymbol{v}_i$ in that direction or subtracts $\boldsymbol{v}_i\cdot\hat{\boldsymbol{n}}_i^{(b)}$), replaces it by adding $v_i^{(b)}$, and then converts to vector in direction $\hat{\boldsymbol{n}}_i^{(b)}$. The second form writes out components and derives outer product matrix $\hat{\boldsymbol{n}}_i^{(b)}\otimes\hat{\boldsymbol{n}}_i^{(b)}$. We can now write:

$$\boldsymbol{v}_i^* = \big(\mathsf{I} - \hat{\boldsymbol{n}}_i^{(b)}\otimes\hat{\boldsymbol{n}}_i^{(b)}\delta_{ib}\big)\boldsymbol{v}_i + v_i^{(b)}\hat{\boldsymbol{n}}_i^{(b)}\delta_{ib}$$

where $\delta_{ib} = 1$ if node i has boundary condition b and zero otherwise. A node can have more than one boundary condition if the $\hat{\boldsymbol{n}}_i^{(b)}$ are orthogonal (and parallel could probably be handled) by changing boundary condition terms to sum of boundary conditions on node i.

We need to find $d\Omega/d\boldsymbol{v}_j$, which is $\nabla\Omega$ with respect to velocity components (and will be a vector) and find that for the three terms above in Ω. The first term is zero because $\boldsymbol{V}_p$ is independent of $\boldsymbol{v}_j$. For the second term, only the $i = j$ term is non zero leading to

$$\frac{d}{d\boldsymbol{v}_j}\big(2S_{pj}\boldsymbol{V}_p\cdot\boldsymbol{v}_j^*\big) = 2S_{pj}\big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)}\otimes\hat{\boldsymbol{n}}_j^{(b)}\delta_{jb}\big)\boldsymbol{V}_p$$

For the last term, we consider two cases for non-zero terms. The first is for $k = j = i$ giving

$$\frac{d}{d\boldsymbol{v}_j}\big(S_{pj}S_{pj}\boldsymbol{v}_j^*\cdot\boldsymbol{v}_j^*\big) = 2S_{pj}S_{pj}\big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)}\otimes\hat{\boldsymbol{n}}_j^{(b)}\delta_{jb}\big)\boldsymbol{v}_j^*$$

The other cases are all the terms where $k = j$, but $i \neq j$ (and two of them for each case). The only term is when $k = j$ or

$$\frac{d}{d\boldsymbol{v}_j}\big(2\sum_{i\neq j}S_{pj}S_{pi}\boldsymbol{v}_j^*\cdot\boldsymbol{v}_i^*\big) = 2\sum_{i\neq j}S_{pj}S_{pj}\big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)}\otimes\hat{\boldsymbol{n}}_j^{(b)}\delta_{jb}\big)\boldsymbol{v}_i^*$$

Combining these terms gets sum that includes $i = j$. The net result of setting $d\Omega/d\boldsymbol{v}_j = 0$ is

$$\sum_i \sum_p M_p S_{pj} S_{pi} \big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)} \otimes \hat{\boldsymbol{n}}_j^{(b)} \delta_{jb}\big) \Big(\big(\mathsf{I} - \hat{\boldsymbol{n}}_i^{(b)} \otimes \hat{\boldsymbol{n}}_i^{(b)} \delta_{ib}\big) \boldsymbol{v}_i + v_i^{(b)} \hat{\boldsymbol{n}}_i^{(b)} \delta_{ib} \Big)$$
$$= \sum_p M_p S_{pj} \big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)} \otimes \hat{\boldsymbol{n}}_j^{(b)} \delta_{jb}\big) \boldsymbol{V}_p$$

Implementation of this approach looks challenging and does not provide a clear path to a new mass matrix or even to a new lumped mass matrix. Also note that if we redefine mass matrix as

$$\tilde{\mathsf{m}}_{ji} = \sum_p M_p S_{pj} S_{pj} \big(\mathsf{I} - \hat{\boldsymbol{n}}_j^{(b)} \otimes \hat{\boldsymbol{n}}_j^{(b)} \delta_{jb}\big) \big(\mathsf{I} - \hat{\boldsymbol{n}}_i^{(b)} \otimes \hat{\boldsymbol{n}}_i^{(b)} \delta_{ib}\big)$$

then the elements are matrices. A lumped mass matrix of matrices would be

$$\mathsf{m}_i = \sum_p M_p S_{pi} \left(\mathsf{I} - \sum_j S_{pj} \hat{\boldsymbol{n}}_j^{(b)} \otimes \hat{\boldsymbol{n}}_j^{(b)} \delta_{jb} \right) \big(\mathsf{I} - \hat{\boldsymbol{n}}_i^{(b)} \otimes \hat{\boldsymbol{n}}_i^{(b)} \delta_{ib}\big)$$

and each element is a matrix.

1.5.1 Prior notes for BC and Contact

This section are some prior notes before deriving the above approach to minimizing Ω. I am not all of this section is correct and seems to be limited to 1D results in places. The metric to minimize

$$\Omega = \sum_p M_p \big(\boldsymbol{V}_p - \tilde{\boldsymbol{V}}_p\big)^2 = \sum_p M_p \big(\boldsymbol{V}_p - \sum_i S_{pi} \boldsymbol{v}_i - \sum_i S_{pi} \Delta \boldsymbol{v}_i\big)^2 \tag{1.47}$$

$$0 = \frac{d\Omega}{d\boldsymbol{v}_j} = -2 \left(\sum_p M_p \big(\boldsymbol{V}_p - \sum_i S_{pi} \boldsymbol{v}_i - \sum_i S_{pi} \Delta \boldsymbol{v}_i\big) \right) \left(S_{pj} + \sum_k S_{pk} \frac{d\Delta \boldsymbol{v}_k}{d\boldsymbol{v}_j} \right) \tag{1.48}$$

Here $\boldsymbol{v}_i$ are velocities in absence of contact and boundary conditions and $\Delta \boldsymbol{v}_i$ is change in velocity for node i caused by contact and boundary conditions ($\Delta \boldsymbol{v}_i = 0$ for nodes remote from contact and boundary conditions).

$$\sum_p M_p \left(S_{pj} + \sum_k S_{pk} \frac{d\Delta \boldsymbol{v}_k}{d\boldsymbol{v}_j} \right) \sum_i S_{pi} (\boldsymbol{v}_i + \Delta \boldsymbol{v}_i) = \sum_p \left(S_{pj} + \sum_k S_{pk} \frac{d\Delta \boldsymbol{v}_k}{d\boldsymbol{v}_j} \right) M_p \boldsymbol{V}_p \tag{1.49}$$

Begin with only boundary conditions where $\Delta \boldsymbol{v}_k = (v_k^{(BC)} - \boldsymbol{v}_k \cdot \hat{\boldsymbol{n}}^{(BC)}) \hat{\boldsymbol{n}}^{(BC)} \delta_{kb}$ where b is list of nodes with boundary conditions and $v_k^{(BC)}$ is boundary condition on node k in that list in direction $\hat{\boldsymbol{n}}^{(BC)}$. For 1D analysis, we have $d\Delta \boldsymbol{v}_k / d\boldsymbol{v}_j = -\delta_{jk} \delta_{kb}$:

$$\big(1 - \delta_{jb}\big) \sum_i \sum_p M_p S_{pj} S_{pi} (\boldsymbol{v}_i + \Delta \boldsymbol{v}_i) = \big(1 - \delta_{jb}\big) \sum_p S_{jp}^T M_p \boldsymbol{V}_p \tag{1.50}$$

If we set $\boldsymbol{v} = \tilde{\mathsf{m}}^{-1} \boldsymbol{p}$, then above equation gives $\boldsymbol{v}_j$ for $j \notin b$. For $j \in b$, the equation is zero equals zero, but those velocities are controlled by the boundary condition. In other words:

$$\boldsymbol{v}_j = \begin{cases} (\tilde{\mathsf{m}}^{-1} \boldsymbol{p})_j & j \notin b \\ \boldsymbol{v}_j^{(BC)} & j \in b \end{cases} \quad \text{or} \quad \boldsymbol{v} = \tilde{\mathsf{m}}^{-1} \boldsymbol{p} + \Delta \boldsymbol{v} \tag{1.51}$$

where $\Delta\boldsymbol{v}$ is change in velocity due to boundary conditions or contact.

If we interpret grid velocity as extrapolated momentum plus any velocity changes needed for contact or grid velocity conditions, the grid velocity can be rewritten as velocity extrapolation with approximate full mass matrix should change. First, rewrite the mapping matrix as:

$$\boldsymbol{v} = \left(\sum_{\ell=1}^{k} C_{k\ell}\left(\mathsf{S}^{+}\mathsf{S}\right)^{\ell-1}\right)\mathsf{S}^{+}\boldsymbol{V} \tag{1.52}$$

$$= k\mathsf{S}^{+}\boldsymbol{V} + \left(\sum_{\ell=2}^{k} C_{k\ell}\left(\mathsf{S}^{+}\mathsf{S}\right)^{\ell-2}\right)\mathsf{S}^{+}\mathsf{S}\boldsymbol{v}^{(L0)} \tag{1.53}$$

where $\boldsymbol{v}^{(L0)} = \mathsf{S}^{+}\boldsymbol{V}$ is lumped mass matrix velocity ignoring contact and boundary conditions. Now imagine contact (which likely will need to use lumped mass matrix methods) or boundary conditions (which would be more efficient with lumped mass matrix methods) causing a change in lumped grid velocity of $\Delta\boldsymbol{v}^{(L)}$ defined by the lumped grid extrapolation being:

$$\boldsymbol{v}^{(L)} = \mathsf{S}^{+}\boldsymbol{V} + \Delta\boldsymbol{v}^{(L)} = \boldsymbol{v}^{(L0)} + \Delta\boldsymbol{v}^{(L)} \tag{1.54}$$

and assume this calculation should start with $\boldsymbol{v}^{(L)}$ in place of $\boldsymbol{v}^{(L0)}$:

$$\boldsymbol{v} = k\mathsf{S}^{+}\boldsymbol{V} + \left(\sum_{\ell=2}^{k} C_{k\ell}\left(\mathsf{S}^{+}\mathsf{S}\right)^{\ell-2}\right)\left(\mathsf{S}^{+}\mathsf{S}\mathsf{S}^{+}\boldsymbol{V} + \mathsf{S}^{+}\mathsf{S}\Delta\boldsymbol{v}^{(L)}\right) \tag{1.55}$$

$$= \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^{(0)} + \left(\sum_{\ell=2}^{k} C_{k\ell}\left(\mathsf{S}^{+}\mathsf{S}\right)^{\ell-1}\right)\mathsf{m}^{-1}\mathsf{m}\Delta\boldsymbol{v}^{(L)} \tag{1.56}$$

$$= \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^{(0)} + \left(\tilde{\mathsf{m}}_k^{-1} - k\mathsf{m}^{-1}\right)\Delta\boldsymbol{p}^{(L)} \tag{1.57}$$

A different approach seems to get a different answer. Rewrite the pseudo-inverse as:

$$\tilde{\mathsf{S}}_k^{+} = \mathsf{S}^{+} + \left(\mathsf{I}_n + (\mathsf{I}_n - \mathsf{S}^{+}\mathsf{S}) + \cdots + (\mathsf{I}_n - \mathsf{S}^{+}\mathsf{S})^{k-2}\right)\mathsf{S}^{+}(\mathsf{I}_N - \mathsf{S}\mathsf{S}^{+}) \tag{1.58}$$

$$= \mathsf{S}^{+} + \tilde{\mathsf{S}}_{k-1}^{+}(\mathsf{I}_N - \mathsf{S}\mathsf{S}^{+}) \tag{1.59}$$

The grid velocity calculation becomes

$$\boldsymbol{v} = \mathsf{S}^{+}\boldsymbol{V} + \tilde{\mathsf{S}}_{k-1}^{+}(\boldsymbol{V} - \mathsf{S}\mathsf{S}^{+}\boldsymbol{V}) = \mathsf{S}^{+}\boldsymbol{V} + \tilde{\mathsf{S}}_{k-1}^{+}(\boldsymbol{V} - \mathsf{S}\boldsymbol{v}^{(L0)}) \tag{1.60}$$

Again, we start with $\boldsymbol{v}^{(L)}$ in place of $\boldsymbol{v}^{(L0)}$:

$$\boldsymbol{v} = \mathsf{S}^{+}\boldsymbol{V} + \tilde{\mathsf{S}}_{k-1}^{+}(\boldsymbol{V} - \mathsf{S}\mathsf{S}^{+}\boldsymbol{V} - \mathsf{S}\Delta\boldsymbol{v}^{(L)}) \tag{1.61}$$

$$= \tilde{\mathsf{S}}_k^{+}\boldsymbol{V} - \tilde{\mathsf{S}}_{k-1}^{+}\mathsf{S}\Delta\boldsymbol{v}^{(L)} \tag{1.62}$$

$$= \tilde{\mathsf{m}}_k^{-1}\mathsf{S}^{T}\mathsf{M}\boldsymbol{V} - \tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{S}^{T}\mathsf{M}\mathsf{S}\Delta\boldsymbol{v}^{(L)} \tag{1.63}$$

$$= \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^{(0)} - \tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{S}^{T}\mathsf{M}\mathsf{S}\mathsf{m}^{-1}\Delta\boldsymbol{p}^{(L)} \tag{1.64}$$

$$= \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^{(0)} - \tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{S}^{+}\mathsf{S}\Delta\boldsymbol{p}^{(L)} \tag{1.65}$$

The new terms can possible be rewritten:

$$\tilde{\mathsf{m}}_k^{-1} - \tilde{\mathsf{m}}_{k-1}^{-1} = (\mathsf{I}_n - \mathsf{S}^{+}\mathsf{S})^{k-1}\mathsf{m}^{-1} \tag{1.66}$$

$$= \mathsf{m}^{-1} - \left(\sum_{\ell=1}^{k-1}\binom{k-1}{\ell}(-1)^{\ell+1}(\mathsf{S}^{+}\mathsf{S})^{\ell-1}\mathsf{m}^{-1}\right)\mathsf{S}^{+}\mathsf{S} \tag{1.67}$$

$$= \mathsf{m}^{-1} - \tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{S}^{+}\mathsf{S} \tag{1.68}$$

$$-\tilde{\mathsf{m}}_{k-1}^{-1}\mathsf{S}^{+}\mathsf{S} = \tilde{\mathsf{m}}_k^{-1} - \tilde{\mathsf{m}}_{k-1}^{-1} - \mathsf{m}^{-1} \tag{1.69}$$

Then

$$\boldsymbol{v} \quad = \quad \tilde{\mathsf{m}}_k^{-1}\left(\boldsymbol{p}^{(0)} + \Delta\boldsymbol{p}^{(L)}\right) - \left(\tilde{\mathsf{m}}_{k-1}^{-1} + \mathsf{m}^{-1}\right)\Delta\boldsymbol{p}^{(L)} \tag{1.70}$$

The contact section below seems to give a third answer. Which is correct? The new incremental method in Section 1.4.3 is the preferred approach.

1.6 Components of Approximate Full Mass Matrix Inverse

Give full mass matrix inverse of

$$\tilde{\mathsf{m}}_k^{-1} = \left(\sum_{\ell=1}^{k} C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\mathsf{m}^{-1} \tag{1.71}$$

can we evaluate individual components (if needed in the algorithm). We define vector

$$\boldsymbol{m}^{(j)} = \left((\tilde{\mathsf{m}}_k)_{1j}^{-1}, (\tilde{\mathsf{m}}_k)_{2j}^{-1}, \ldots, (\tilde{\mathsf{m}}_k)_{nj}^{-1}\right) = \tilde{\mathsf{m}}_k^{-1}\boldsymbol{v} \tag{1.72}$$

where elements of vector $\boldsymbol{v}$ are $v_i = \delta_{ij}$ and we have the components of the inverse from $(\tilde{\mathsf{m}}_k)_{ij}^{-1} = \boldsymbol{m}_i^{(j)}$. We can write

$$\boldsymbol{m}^{(j)} = \left(\sum_{\ell=1}^{k} C_{k\ell}\left(\mathsf{S}^+\mathsf{S}\right)^{\ell-1}\right)\boldsymbol{v} \tag{1.73}$$

where vector $\boldsymbol{v}$ is redefined with components $v_i = \delta_{ij}/m_j$. This result is identical to XPIC calculations for velocity. We can thus extract for j of inverse matrix by seeding nodal velocities with $v_i^* = k\delta_{ij}/m_j$ and using XPIC methods to find final $\boldsymbol{m}^{(j)} = \boldsymbol{v}^*$.

1.7 Some Extrapolation Results

Imagine some particle quantity summed over all particles weighted by the particle mass:

$$Q_{total} = \sum_p M_p Q_p = \boldsymbol{Q}\cdot\boldsymbol{M} \tag{1.74}$$

Multiply by $1 = \sum_i S_{pi}$, insert definition of $m_i S_{ip}^+ = M_p S_{pi}$, and nodal mass $m_i = \sum_q M_q S_{qi}$ to get

$$Q_{total} = \sum_p\sum_i S_{pi}M_pQ_p = \sum_p\sum_i m_i S_{ip}^+ Q_p = \sum_p\sum_q\sum_i S_{qi}S_{ip}^+ M_q Q_p$$

Converting to vectors gives

$$Q_{total} = \sum_p\sum_q (\mathsf{SS}^+)_{qp}Q_pM_q = \sum_q(\mathsf{SS}^+\boldsymbol{Q})_q M_q = \mathsf{SS}^+\boldsymbol{Q}\cdot\boldsymbol{M}$$

Equating two results for Q_{total} leads to

$$(\mathsf{I}_N - \mathsf{SS}^+)\boldsymbol{Q}\cdot\boldsymbol{M} = 0$$

This result holds for any particle quantity — although $\mathsf{SS}^+ \neq I_N$, when summed over all particles the net result is zero. Because $(\mathsf{I}_N - \mathsf{SS}^+)\boldsymbol{Q}$ is also a particle quantity, we can prove by induction that

$$(\mathsf{I}_N - \mathsf{SS}^+)^k\boldsymbol{Q}\cdot\boldsymbol{M} = 0 \tag{1.75}$$

Because any grid quantity can be extrapolated to particles, $\mathsf{S}\boldsymbol{q}$ is a particle quantity leading to a result for grid quantities:

$$\mathsf{S}(\mathsf{I}_n - \mathsf{S}^+\mathsf{S})^k \boldsymbol{q} \cdot \boldsymbol{M} = 0 \tag{1.76}$$

These results can be used to show that total linear momentum is always conserved when extrapolating back and forth between the grid and particles for all FLIP, XPIC(k), and FMPM(k) extrapolations.

Chapter 2

The MPM Algorithm - Single Material Mode

This chapter goes through each MPM task, writes it using vector and matrix notation, and then discusses changes needed to implement full mass matrix interpretation of XPIC(m) denoted as FMPM. The tasks are for single-velocity field mode. Dealing with cracks, multiple materials, and contact is covered in subsequent chapters.

2.1 MPM Time Step Tasks

The section outlines the various tasks in `OSParticulas` and where changes are needed to implement full mass matrix.

2.1.1 Extrapolate Mass and Momentum to the Grid

The first task is to extrapolate mass and momenta to the grid using

$$\boldsymbol{p}_i = \sum_p S_{ip} \boldsymbol{P}_p \quad \text{or} \quad \boldsymbol{p} = \mathsf{S}^T \boldsymbol{P} = \mathsf{S}^T \mathsf{M} \boldsymbol{V} \tag{2.1}$$

with particle mass matrix $\mathsf{M} = \text{diag}(\boldsymbol{M})$ and nodal lumped mass from $\boldsymbol{m} = \mathsf{S}^T \boldsymbol{M}$. The full mass matrix is

$$\tilde{\mathsf{m}} = \mathsf{S}^T \mathsf{M} \mathsf{S} \tag{2.2}$$

But it is neither calculated nor stored.

Post Initial Extrapolations

The tasks done in `OSParticulas` are:

- Find total nodal mass by summing masses from material and crack velocity field and identify material and crack contact nodes.
- Adjust momenta on multimaterial and crack contact nodes. This task is not done in single velocity field MPM.
- Change nodal nodal momenta to match velocity boundary conditions. The details are given in section 2.4.

Althought steps 2 and 3 are done every time step in `OSParticulas`, I think some can be skipped depending on update method (*e.g.*, velocity BCs not needed if using `USL`± update methods).

2.1.2 Update Strains First

If being done, this task updates the particle strains by extrapolating velocity gradient from the grid to the particles, but this should now be based on velocities using full mass matrix. The new task is to find grid velocity using

$$\boldsymbol{v}(k) = \tilde{\mathsf{S}}_k^+ \boldsymbol{V} = \tilde{\mathsf{m}}_k^{-1} \mathsf{S}^T \mathsf{M} \boldsymbol{V} = \left(\sum_{\ell=1}^{k} C_{k\ell} \left(\mathsf{S}^+ \mathsf{S} \right)^{\ell-1} \right) \boldsymbol{v}^{(L)} \tag{2.3}$$

where $\boldsymbol{v}_i^{(L)} = \boldsymbol{p}_i'/m_i$ are velocities from for momenta corrected for contact (not done in single velocity mode) and boundary conditions (see section 2.4). This calculation is identical to XPIC(k) task of finding $\boldsymbol{v}(k)$. The XPIC(k) style implementation is

$$\boldsymbol{v}(k) \quad = \quad \sum_{\ell=1}^{k} \boldsymbol{v}_\ell \qquad \text{where} \qquad \boldsymbol{v}_\ell = C_{k\ell} (\mathsf{S}^+ \mathsf{S})^{\ell-1} \boldsymbol{v}^{(L)} \tag{2.4}$$

As a recursion relation:

$$\boldsymbol{v}_\ell = C_{k,\ell-1} \frac{C_{k\ell}}{C_{k,\ell-1}} (\mathsf{S}^+ \mathsf{S})(\mathsf{S}^+ \mathsf{S})^{\ell-2} \boldsymbol{v}^{(L)} = \frac{C_{k\ell}}{C_{k,\ell-1}} (\mathsf{S}^+ \mathsf{S}) \boldsymbol{v}_{\ell-1} \quad \text{seeded with} \quad \boldsymbol{v}_1 = k \boldsymbol{v}^{(L)} \tag{2.5}$$

From `Extrap.pdf` notes after a sign addition to keep -1 term in definition:

$$\frac{C_{k\ell}}{C_{k,\ell-1}} = -\frac{k-\ell+1}{k} \tag{2.6}$$

The revised FMPM methods described above is the preferred approach by summing $\Delta \boldsymbol{v}(\ell)$ instead of $\boldsymbol{v}_\ell$.

$$\boldsymbol{v}(k) = \sum_{\ell=1}^{k} \Delta \boldsymbol{v}(\ell) \quad \text{where} \quad \Delta \boldsymbol{v}(\ell) = \boldsymbol{v}(\ell) - \boldsymbol{v}(\ell-1) = \mathsf{A}^{\ell-1} \boldsymbol{v}(1) = \mathsf{A} \Delta \boldsymbol{v}(\ell-1)$$

Inserting A gives

$$\Delta \boldsymbol{v}(\ell) = \Delta \boldsymbol{v}(\ell-1) - \mathsf{S}^+ \mathsf{S} \Delta \boldsymbol{v}(\ell-1)$$

which is effectively same code as previously used to find $\boldsymbol{v}_\ell$ (and even slightly simpler). See Section 1.4.3 for full implementation.

Once velocities are found, extrapolate to particle using gradient shape functions:

$$\nabla \boldsymbol{v}_p = \sum_i \boldsymbol{v}_i \otimes \boldsymbol{G}_{pi} = \sum_i \boldsymbol{v}_i^T \boldsymbol{G}_{pi} \tag{2.7}$$

where vectors are row vectors and transposing switches between row and column vectors. Define gradient shape function matrix as $N \times n$ matrix G where each element is a column vector:

$$(\mathsf{G})_{pi} = (\boldsymbol{G}_{pi})^T = \left(\frac{\int \chi_p(\boldsymbol{x}) \nabla N_i(\boldsymbol{x}) d\boldsymbol{x}}{\int \chi_p(\boldsymbol{x}) d\boldsymbol{x}} \right)^T \tag{2.8}$$

This extrapolation can be cast in matrix form as

$$\nabla \boldsymbol{v}^T = \sum_i (\mathsf{G})_{pi} \boldsymbol{v}_i \quad \text{or} \quad \nabla \boldsymbol{v}^T = \mathsf{G} \boldsymbol{v} = \mathsf{G} \tilde{\mathsf{m}}_k^{-1} \boldsymbol{p}' \tag{2.9}$$

where $\nabla \boldsymbol{v}^T$ is vector of transposes of the particle velocity gradient tensors.

2.1.3 Extrapolate Forces

The force extrapolation to node i is:

$$\boldsymbol{f}_i = -\sum_p \frac{M_p \boldsymbol{\tau}_p (\mathsf{G})_{pi}}{\rho_{p,0}} + \boldsymbol{f}_{i,T} + \sum_p \left(M_p S_{ip} \boldsymbol{b}_p + \boldsymbol{F}_p S_{ip} \right) \tag{2.10}$$

The first term is internal force and the rest are external forces (tractions, body forces, and particle forces). Note that $\boldsymbol{\tau}_p$ is Kirchoff stress tensor. In matrix form (and because $\boldsymbol{\tau}_p$ is symmetric):

$$\boldsymbol{f} = -\mathsf{M}\mathsf{G}^T \tilde{\boldsymbol{\tau}} + \boldsymbol{f}_T + \mathsf{M}\mathsf{S}^T \boldsymbol{b} + \mathsf{S}^T \boldsymbol{F} \tag{2.11}$$

where $(\mathsf{G}^T)_{ip} = \boldsymbol{G}_{pi}$ are row vectors and $\tilde{\boldsymbol{\tau}}$ is vector with $\boldsymbol{\tau}_p / \rho_{p,0}$ in element p (and $\rho_{p,0}$ is undeformed particle density). This task is unchanged when using full mass matrix.

Post Force Extrapolation

The tasks done after extrapolating forces (to finish force calculations) in `OSParticuals` are:

- Traction force are done here in code (*i.e.*, the $\boldsymbol{f}_T$ terms in force extrapolations).
- Add tractions on crack surfaces from cohesive laws (can be imagined as part of traction forces)
- Add body forces and optional add damping proportional to $\boldsymbol{v}_i$ or $\boldsymbol{f}_i$. Body forces done here assume they depend only on grid position on not on state of the particle.
- Change forces to implement velocity boundary conditions (see section 2.4). Not that here we change force while prior to strain updates, we change momenta.

2.1.4 Update Grid Momenta

Update grid momenta using contact forces:

$$\boldsymbol{p}^+ = \boldsymbol{p} + \boldsymbol{f} \Delta t \tag{2.12}$$

Post Momentum Update

The tasks after the momentum update are:

- Crack and multimaterial contact calculations. These will need to use lumped mass matrix methods. This task is not done in single velocity field MPM.
- Reimpose velocity boundary conditions in particle momenta. In regular MPM, this step is done to handle cases where contact calculations conflicted with boundary conditions. In FMPM, this calculation may or may not be needed and will change velocities instead of momenta or forces (see section 2.4).

2.1.5 Update Particles

The particle update (not including damping for now) uses PIC or

$$\boldsymbol{V}^{(n+1)} = \mathsf{S}\tilde{\mathsf{m}}_k^{-1} \boldsymbol{p}^+ = \mathsf{S}(\tilde{\mathsf{m}}_k^{-1} \boldsymbol{p} + \tilde{\mathsf{m}}_k^{-1} \boldsymbol{f} \Delta t) = \mathsf{S}(\boldsymbol{v}(k) + \tilde{\mathsf{m}}_k^{-1} \boldsymbol{f} \Delta t) \tag{2.13}$$

We find acceleration $\boldsymbol{a} = \tilde{\mathsf{m}}_k^{-1} \boldsymbol{f}$ using an XPIC task. More details and the position update are given in section 2.3.

2.1.6 Update Strains Last

For USL–and USAVG–, this tasks can used the final velocities found in the particle update tasks. If using USL+ or USAVG+, FMPM will need to do contact, impose boundary conditions, and finally re-use XPIC(k) task on re-extrapoalted momenta prior to the strain update.

2.1.7 Move Cracks Tasks

Use the correct velocities, but not done in single-material mode MPM.

2.2 Bare Bones MPM Algorithm

The following is bare-bones definition of MPM algorithm including particle tractions and velocity boundary conditions. It omits cracks, contact/multimaterial mode, damping, and coupled transport tasks.

1. Extrapolate mass and momentum to the grid:

$$\boldsymbol{m}\mathsf{S}^T\boldsymbol{M} \qquad \text{and} \qquad \boldsymbol{p} = \mathsf{S}^T\mathsf{M}\boldsymbol{V} \tag{2.14}$$

2. For USF or USAVG± update options, update particle stress and strains as explained below.

3. Extrapolate forces to the grid including forces due to traction boundary conditions and body forces:

$$\boldsymbol{f} = -\mathsf{M}\mathsf{G}^T\tilde{\boldsymbol{\tau}} + \boldsymbol{f}_T + \mathsf{M}\mathsf{S}^T\boldsymbol{b} + \mathsf{S}^T\boldsymbol{F} \tag{2.15}$$

4. Update grid momenta

$$\boldsymbol{p}^+ = \boldsymbol{p} + \boldsymbol{f}\Delta t \tag{2.16}$$

5. Find grid velocity using $\boldsymbol{v}^+ = \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+$ and then update particle velocity and position using

$$\boldsymbol{V}^{(n+1)} = \mathsf{S}\boldsymbol{v}^+ \qquad \text{and} \qquad \boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \frac{1}{2}\left(\boldsymbol{V}^{(n+1)} + \boldsymbol{V}^{(n)}\right)\Delta t \tag{2.17}$$

6. For USL± or USAVG± update options:

   (a) For USL– or USAVG–, this update can use $\boldsymbol{v}^+$ just calculated in the particle update tasks and then update particle stress and strains as explained below but skip the first step.

   (b) For USL+ or USAVG+, re-extrapolate $\boldsymbol{p} = \mathsf{S}^T\mathsf{M}\boldsymbol{V}$ and then update particle stress and strains as explained below.

Once or twice per time step, the current grid velocity are used as input to update stresses and strains on the particle. The process for this update is:

1. Change current momenta to $\boldsymbol{p}' = \boldsymbol{p} + \boldsymbol{f}^{(BC)}\Delta t$ and find grid velocity from $\boldsymbol{v} = \tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}'$

2. Extrapolate velocity gradient to the particles

$$\nabla\boldsymbol{v}^T = \mathsf{G}\boldsymbol{v} \tag{2.18}$$

3. Update stress and strains using particle's constitutive law. Note that $\nabla\boldsymbol{v}^T$ is spatial gradient. Also note that for USAVG±, the update uses $\Delta t/2$ for the time step.

2.3 Particle Update Details

We can express particle velocity and position update in a generic form as

$$\boldsymbol{V}^{(n+1)} = \boldsymbol{V}^{(n)} + \mathbb{A}^{(n)}\Delta t \tag{2.19}$$

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \mathbb{V}^{(n)}\Delta t + \frac{1}{2}\mathbb{A}^{(n)}(\Delta t)^2 \tag{2.20}$$

All MPM methods can be expressed in this form. For example, give any equations for particle velocity and position updates, we can derive that method's implied particle acceleration and initial velocity as:

$$\mathbb{A}^{(n)}\Delta t = \boldsymbol{V}^{(n+1)} - \boldsymbol{V}^{(n)} \tag{2.21}$$

$$\mathbb{V}^{(n)}\Delta t = \boldsymbol{X}^{(n+1)} - \boldsymbol{X}^{(n)} - \frac{1}{2}\left(\boldsymbol{V}^{(n+1)} - \boldsymbol{V}^{(n)}\right)\Delta t \tag{2.22}$$

Note that the net Lagrangian velocity imposed by the position update is

$$\left(\frac{\Delta \boldsymbol{X}}{\Delta t}\right) = \mathbb{V}^{(n)} + \frac{1}{2}\mathbb{A}^{(n)}\Delta t \tag{2.23}$$

while the average Lagrangian velocity imposed by the velocity update is

$$\langle \boldsymbol{V}\rangle_{\boldsymbol{X}} = \boldsymbol{V}^{(n)} + \frac{1}{2}\mathbb{A}^{(n)}\Delta t \tag{2.24}$$

In other words, for methods with $\mathbb{V}^{(n)} \neq \boldsymbol{V}^{(n)}$, the method will propagate an error between expected Lagrangian updates for particle position and velocity of:

$$\text{error} \propto \left(\frac{\Delta \boldsymbol{X}}{\Delta t}\right) - \langle \boldsymbol{V}\rangle_{\boldsymbol{X}} = \mathbb{V}^{(n)} - \boldsymbol{V}^{(n)} \tag{2.25}$$

2.3.1 FMPM Updates

The update proposed for FMPM(k), reduces to generic form using

$$\mathbb{A}^{(n)}\Delta t = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+ - \boldsymbol{V}^{(n)} \qquad \text{and} \qquad \mathbb{V}^{(n)} = \boldsymbol{V}^{(n)} \tag{2.26}$$

The error in Lagrangian updates is zero.

2.3.2 Prior FLIP MPM Updates

Now contrast this to standard FLIP MPM updates of:

$$\boldsymbol{V}^{(n+1)} = \boldsymbol{V}^{(n)} + \mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t \tag{2.27}$$

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \left(\mathsf{S}\mathsf{m}^{-1}\boldsymbol{p}^+ - \frac{\psi}{2}\mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t\right)\Delta t \tag{2.28}$$

In other words, find acceleration and updated velocity on the grid using lumped mass matrix followed by velocity update using acceleration extrapolated to the particle and position update using updated velocity and a second-order term involving acceleration. The original MPM papers (and most papers in the literature) ignore the second order term in position update because it is small. This common update is recovered by setting $\psi = 0$. Using $\psi = 1$ illustrates changes by including a second order term.

The standard MPM FLIP form reduces to generic form using:

$$\mathbb{A}^{(n)}\Delta t = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t \quad \text{and} \quad \mathbb{V}^{(n)} = \mathsf{S}\mathsf{m}^{-1}\left(\frac{(1-\psi)\boldsymbol{p}^{+}+(1+\psi)\boldsymbol{p}}{2}\right) \tag{2.29}$$

The error in Lagrangian updates become

$$\text{error} \propto \left(\frac{\Delta \boldsymbol{X}}{\Delta t}\right) - \langle \boldsymbol{V}\rangle_{\boldsymbol{X}} = \mathsf{S}\mathsf{m}^{-1}\left(\frac{(1-\psi)\boldsymbol{p}^{+}+(1+\psi)\boldsymbol{p}}{2}\right) - \boldsymbol{V}^{(n)} \tag{2.30}$$

If $\psi = 0$ (which is most common in the literature)

$$\mathbb{A}^{(n)}\Delta t = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t \qquad \text{and} \qquad \mathbb{V}^{(n)} = \mathsf{S}\mathsf{m}^{-1}\left(\frac{\boldsymbol{p}^{+}+\boldsymbol{p}}{2}\right) \tag{2.31}$$

Extending to second order in position update with $\psi = 1$ gives

$$\mathbb{A}^{(n)}\Delta t = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t \qquad \text{and} \qquad \mathbb{V}^{(n)} = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{p}, \tag{2.32}$$

Both values of ψ result in Lagrangian update error, but this was not an error in original MPM development. One can easily eliminate that error by choosing $\mathbb{V}^{(n)} = \boldsymbol{V}^{(n)}$ (as used in FMPM update). This choice would change the standard FLIP MPM second-order update for position to be

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \left(\boldsymbol{V}^{(n)} + \frac{1}{2}\mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t\right)\Delta t \tag{2.33}$$

Although this eliminates Lagrangian update errors (because $\mathbb{V}^{(n)} = \boldsymbol{V}^{(n)}$), it gives very poor results. The solution to stabilize FLIP update was to replace velocity used in the position update with the smooth velocity defined above (see Eq. (2.29)). This suggestions was likely made in early days of FLIP updates (by Brackbilll) and perhaps has been forgotten that it was a trick to improve updates rather than ideal choice based on Lagrangian particles. An advantage of FMPM is one no longer needs an *ad hoc* change in velocity used in particle update.

2.3.3 Prior XPIC Updates

Prior to recognizing that XPIC(m) calculations done on grid momenta is the same as using $\tilde{\mathsf{m}}_{k=m}^{-1}$ on those momenta, the prior XPIC($m = k$) update now using mass matrix notation was proposed as:

$$\boldsymbol{V}^{(n+1)} = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p} + \mathsf{S}\mathsf{m}^{-1}\boldsymbol{f}\Delta t \tag{2.34}$$

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \left[\mathsf{S}\mathsf{m}^{-1}\boldsymbol{p} + \frac{\boldsymbol{V}^{(n+1)} - \boldsymbol{V}^{(n)}}{2}\right]\Delta t \tag{2.35}$$

These updates reduce to generic form using:

$$\mathbb{A}^{(n)}\Delta t = \mathsf{S}\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^{+} + \mathsf{S}\left(\mathsf{m}^{-1} - \tilde{\mathsf{m}}_k^{-1}\right)\boldsymbol{f}\Delta t - \boldsymbol{V}^{(n)} \quad \text{and} \quad \mathbb{V}^{(n)} = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{p} \tag{2.36}$$

with Lagrangian update error

$$\text{error} \propto \left(\frac{\Delta \boldsymbol{X}}{\Delta t}\right) - \langle \boldsymbol{V}\rangle_{\boldsymbol{X}} = \mathsf{S}\mathsf{m}^{-1}\boldsymbol{p} - \boldsymbol{V}^{(n)} \tag{2.37}$$

Note that XPIC(m) is identical to standard FLIP (with $\psi = 1$) except uses a different effective acceleration. That acceleration reduces noise to help stabilize FLIP simulations. The new FMPM(k) updates improves further by using $\tilde{\mathsf{m}}_k$ in place of m in $\mathbb{A}^{(n)}$ and setting $\mathbb{V}^{(n)} = \boldsymbol{V}^{(n)}$.

2.3.4 Various PIC Updates

XPIC(1) update reduces to a PIC method:

$$\boldsymbol{V}^{(n+1)} = \mathsf{Sm}^{-1}\boldsymbol{p}^{+} \quad \text{and} \quad \boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \left[\mathsf{Sm}^{-1}\boldsymbol{p} + \frac{\mathsf{Sm}^{-1}\boldsymbol{p}^{+} - \boldsymbol{V}^{(n)}}{2}\right]\Delta t \tag{2.38}$$

These reduce to generic form

$$\mathbb{A}^{(n)}\Delta t = \mathsf{Sm}^{-1}\boldsymbol{p}^{+} - \boldsymbol{V}^{(n)} \qquad \text{and} \qquad \mathbb{V}^{(n)} = \mathsf{Sm}^{-1}\boldsymbol{p} \tag{2.39}$$

In contrast, FMPM(1) reduces to new PIC method:

$$\boldsymbol{V}^{(n+1)} = \mathsf{Sm}^{-1}\boldsymbol{p}^{+} \quad \text{and} \quad \boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \left[\frac{\mathsf{Sm}^{-1}\boldsymbol{p}^{+} + \boldsymbol{V}^{(n)}}{2}\right]\Delta t \tag{2.40}$$

These reduce to generic form

$$\mathbb{A}^{(n)}\Delta t = \mathsf{Sm}^{-1}\boldsymbol{p}^{+} - \boldsymbol{V}^{(n)} \qquad \text{and} \qquad \mathbb{V}^{(n)} = \boldsymbol{V}^{(n)} \tag{2.41}$$

Calculations show that PIC derived from FMPM(1) is more stable than the one derived from XPIC(1) (especially for problems where PIC works well such as MMS with linear velocities).

Some papers (in animation, such as Stomakhin, *et al.*) suggested blending PIC and FLIP, but they used the wrong-headed, position update in their FLIP implementation of

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \boldsymbol{V}^{(n+1)}\Delta t$$

instead of standard first order FLIP update ($\psi = 0$) of

$$\boldsymbol{X}^{(n+1)} = \boldsymbol{X}^{(n)} + \mathsf{Sm}^{-1}\boldsymbol{p}^{+}\Delta t$$

Problems with this wrong-headed, position update were pointed out by Brackbill. The problems with that update probably explains why they needed to blend PIC and FLIP, but they blended only the velocity update and not the position update (or mistakenly thought FLIP and PIC position updates are the same). For a fraction α of FLIP, their complete updates are

$$\begin{aligned}
\boldsymbol{V}^{(n+1)} &= \alpha\left(\boldsymbol{V}^{(n)} + \mathsf{Sm}^{-1}\boldsymbol{f}\Delta t\right) + (1-\alpha)\mathsf{Sm}^{-1}\boldsymbol{p}^{+} = \alpha\boldsymbol{V}^{(n)} + \mathsf{Sm}^{-1}\left(\boldsymbol{p}^{+} - \alpha\boldsymbol{p}\right)\\
\boldsymbol{X}^{(n+1)} &= \boldsymbol{X}^{(n)} + \boldsymbol{V}^{(n+1)}\Delta t
\end{aligned}$$

These correspond to general form

$$\begin{aligned}
\mathbb{A}^{(n)}\Delta t &= \mathsf{Sm}^{-1}\left(\boldsymbol{p}^{+} - \alpha\boldsymbol{p}\right) - (1-\alpha)\boldsymbol{V}^{(n)}\\
\mathbb{V}^{(n)} &= \frac{\boldsymbol{V}^{(n)} + \boldsymbol{V}^{(n+1)}}{2} = \frac{(1+\alpha)\boldsymbol{V}^{(n)} + \mathsf{Sm}^{-1}\left(\boldsymbol{p}^{+} - \alpha\boldsymbol{p}\right)}{2}
\end{aligned}$$

Note that $\alpha = 1$ gives a poor version of FLIP with acceleration correct, but effective velocity changed to

$$\mathbb{V}^{(n)} = \boldsymbol{V}^{(n)} + \frac{1}{2}\mathsf{Sm}^{-1}\boldsymbol{f}\Delta t$$

Picking $\alpha = 0$ gives another variant for PIC with same acceleration as others, but effective velocity changed to

$$\mathbb{V}^{(n)} = \frac{\boldsymbol{V}^{(n)} + \mathsf{Sm}^{-1}\boldsymbol{p}^{+}}{2}$$

Table 2.1: Inputs are velocities and acceleration extrapolated to the particles ($v_p = \mathsf{S}v^+$, $v_m = \mathsf{S}v$, $v_s = \mathsf{S}v^*$, $a = \mathsf{S}\mathsf{m}^{-1}f$) and the current time step ($dt = \Delta t$ or $ht = \Delta t/2$). Outputs are updated particle velocity, V_p, and position, X_p.The "M", "A", and"C" in last line are the number of multiplications, additions, and memory copies in each update method.

FLIP(Std)	PIC(Std)	FMPM(0)/XPIC(0)	XPIC(k>0)	FMPM(k>0)
$V_p \mathrel{+}= a\,dt$ $X_p \mathrel{+}= v_p dt$	$V_p = v_p$ $X_p \mathrel{+}= v_p dt$	$dV = a\,dt$ $V_p \mathrel{+}= dV$ $X_p \mathrel{+}= (v_p - 0.5dV)dt$	$dV = -V_p$ $V_p = k(v_m - v_s) + a\,dt$ $dV \mathrel{+}= V_p$ $X_p \mathrel{+}= (v_m + 0.5dV)dt$	$V_p = v_p$ $X_p \mathrel{+}= (V_p + v_p)ht$
a, v_p	v_p	a, v_p	a, v_m, v_s	v_p
2M, 2A	1M, 1A, 1C	3M, 3A, 1C	4M, 5A, 2C	1M, 2A, 1C

2.3.5 Particle Update Code Comparison

Table 2.1 gives codes steps required for each update method. The list line gives computation cost in terms of multiplication, additions, and memory copies. XPIC(k) needs most calculations because it needs the non-updated velocity on the grid in v_m. FMPM(k) and XPIC(k) also have computational cost in finding v^+. The table is only the cost after this value is found. Some methods only need some inputs and thus can save cost by not calculating those not needed.

Note that FMPM(k) is more efficient than XPIC(k) because of its more consistent treatment of velocities. Also note that neither PIC(Std) nor FMPM(k) even need acceleration as an input. Notice that neither XPIC(1) nor FMPM(1) reduce to PIC(Std). Instead they give improved forms of PIC with FMPM(1) the best. Finally, FMPM(0)/XPIC(0) is only different than FLIP(Std) because it is using a second order position update.

2.4 Velocity Boundary Conditions - Background

This section outlines grid BC methods in MPM and in the first FMPM paper. These initial options had issues as the FMPM order increased. See Section 2.5 for revised methods that seems to resolve the problems. One way to view velocity boundary conditions is that they add a force to each node, $\boldsymbol{f}^{(BC)}$, such that the updated momentum changes to:

$$\boldsymbol{p}^{+\prime} = \boldsymbol{p} + (\boldsymbol{f} + \boldsymbol{f}^{(BC)})\Delta t = \boldsymbol{p}^+ + \boldsymbol{f}^{(BC)}\Delta t \tag{2.42}$$

and the final velocities are given by:

$$\boldsymbol{v}^+ = \tilde{\mathsf{m}}^{-1}\boldsymbol{p}^+ = \tilde{\mathsf{m}}^{-1}(\boldsymbol{p} + \boldsymbol{f}\Delta t) \quad \text{and} \quad \boldsymbol{v}^{+\prime} = \tilde{\mathsf{m}}^{-1}\boldsymbol{p}^{+\prime} \tag{2.43}$$

satisfy boundary conditions on controlled nodes. The boundary conditions cause $\boldsymbol{v}^+$ to change to $\boldsymbol{v}^{+\prime}$ which by full mass matrix is given by

$$\Delta\boldsymbol{v} = \boldsymbol{v}^{+\prime} - \boldsymbol{v}^+ = \tilde{\mathsf{m}}^{-1}\boldsymbol{f}^{(BC)}\Delta t = \tilde{\mathsf{m}}^{-1}\Delta\boldsymbol{p} \implies \Delta\boldsymbol{p} = \tilde{\mathsf{m}}\Delta\boldsymbol{v}$$

where $\Delta\boldsymbol{p}$ is momentum change caused by boundary conditions. Due to off-diagonal elements of $\tilde{\mathsf{m}}$, the momentum change will be on more nodes than just the boundary condition nodes.

To define $\Delta\boldsymbol{v}$, imagine a single node i with possibly multiple velocity boundary conditions b setting the velocity on node i to $v_i^{(b)}$ in direction $\hat{\boldsymbol{n}}^{(b)}$. Commonly a node has a single boundary condition in

one direction, but it could have more than one. When there are more than one, we require that $\hat{\boldsymbol{n}}^{(b)}$ are parallel or perpendicular. Parallel BCs function to superpose velocities. Perpendicular BCs control more directions (maximum of 2 in 2D or 3 in 3D). The imposed velocity changes on nodes with boundary conditions are

$$\left(\tilde{\mathrm{m}}^{-1}\Delta\boldsymbol{p}\right)_i = \Delta\boldsymbol{v}_i = \sum_b \left(v_i^{(b)} - \boldsymbol{v}_i^+ \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)}$$

but if multiple $\hat{\boldsymbol{n}}^{(b)}$ on the same node are parallel, the $\boldsymbol{v}_i^+ \cdot \hat{\boldsymbol{n}}^{(b)}$ is only subtracted for one of them. Multiple BCs on the same node, including some in parallel, can be handled in code to set $\boldsymbol{v}_i^{+\prime}$

let $\boldsymbol{v}^{+\prime} = \boldsymbol{v}^+$
for each node i **with a BC do**
  for each BC b **on node** i **do**
    let $\boldsymbol{v}^{+\prime} -= \left(\boldsymbol{v}^{+\prime} \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)}$
  for each BC b **on node** i **do**
    let $\boldsymbol{v}^{+\prime} += v_i^b \hat{\boldsymbol{n}}^{(b)}$

The first loop subtracts component of original velocity in the $\hat{\boldsymbol{n}}^{(b)}$ direction. If two BC are in the same direction, the second sum will find current $\boldsymbol{v}^{+\prime} \cdot \hat{\boldsymbol{n}}^{(b)} = 0$ so including that term in the sum is acceptable (it just subtracts zero). The second loop superposes the desired velocity conditions.

The mechanical goal of velocity BCs is to apply reaction forces on the grid such that all nodes with velocity BCs change their velocity by $\Delta\boldsymbol{v}_i$. This goal has multiple solutions: 1. Solve for grid forces to give $\Delta\boldsymbol{v}_i$ on nodes with velocity BC by zero on nodes without velocity BCs. 2. Apply $\boldsymbol{f}^{(BC)}$ only to nodes with velocity BCs and only in directions of the BCs. 3. Any combination of 1 and 2. When using full mass matrix methods, all of these option are difficult (and not sure which is correct). If we switch to lumped mass matrix, however, all options can be solved. A lumped mass matrix solutions is

$$\Delta\boldsymbol{p}_i = \Delta\boldsymbol{f}_i^{(BC)}\Delta t = \begin{cases} \sum_b m_i\left(v_i^{(b)} - \boldsymbol{v}_i^+ \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)} & \text{Nodes with BCs} \\ 0 & \text{Nodes without BCs} \end{cases}$$

where $\boldsymbol{a}_i$ is acceleration needed to satisfy boundary conditions. This calculation gets $\Delta\boldsymbol{v}_i$ only on nodes with velocity BCs and applies $\boldsymbol{f}^{(BC)}$ only to nodes with velocity BCs (option 3 is trivially same as 1 and 2).

If full mass matrix methods cannot solve for velocity BCs, we need to approximately include lumped-mass BCs in FMPM. Several options for FMPM were considered first with comments below (note that Section 2.5 has preferred method that was derived after the following attempts).

Lumped Method The simplest approach is to use prior lumped mass matrix methods without modification. The boundary condition force equation simplifies to

$$\boldsymbol{f}^{(L)}\Delta t = \sum_b \Delta\boldsymbol{p} \quad \text{where} \quad \Delta\boldsymbol{p}_j = \left(m_i v_i^{(b)} - \boldsymbol{p}_i^+ \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)}\delta_{ij} \tag{2.44}$$

Remark: The forces are added only to nodes with boundary conditions, but when full mass matrix inverse is used to find grid velocities, those forces will also perturb nodes near the boundary conditions. It might be beneficial to boundary condition performance to distribute its effects of more than just the one boundary condition node.

Remark: The full mass matrix inverse will not project lumped force into boundary condition velocities. As a consequence, force on one boundary condition may perturb velocities on other nodes with boundary conditions. This approach, will only approximate boundary condition velocities in nodes with boundary conditions.

Velocity Method This also-simple approach applies boundary conditions after each calculation of grid velocity using full mass matrix methods. For example, in the particle update, the final updated velocities become sum of updated velocities ignoring boundary conditions with velocities changes imposed by boundary conditions:

$$\boldsymbol{v}^{+\prime} = \boldsymbol{v}^{+} + \sum_{b} \Delta \boldsymbol{v}^{(b)} \quad \text{where} \quad \Delta \boldsymbol{v}_j^{(b)} = \left(v_i^{(b)} - \boldsymbol{v}_i^{+} \cdot \hat{\boldsymbol{n}}^{(b)} \right) \hat{\boldsymbol{n}}^{(b)} \delta_{ij} \tag{2.45}$$

Remark: This approach corresponds to distributing forces over many nodes such that the resulting accelerations ($\boldsymbol{a}^{(b)} = \tilde{\mathsf{m}}_k^{-1} \boldsymbol{f}^{(b)}$) are localized only on nodes with boundary conditions. The grid velocities will exactly match boundary condition values, but nodes without boundary conditions (but near them) will feel no influence of the boundary.

Remark: By doing this calculation after finding grid velocities that ignore boundary condition, this approach avoids the need to ever calculate $\boldsymbol{f}^{(b)}$. One drawback is that reaction force at boundary conditions are not available. If desired for output, the reaction forces could be found using full mass matrix — $\boldsymbol{f}^{(b)} = \tilde{\mathsf{m}} \boldsymbol{a}^{(b)}$ (although maybe this needs to replace $\tilde{\mathsf{m}}$ with $\tilde{\mathsf{m}}_k$?).

Combined Method Another simple option is to combine "Lumped" and "Velocity" methods. This approach spreads effects to nearby nodes, which might be appropriate, and exactly satisfies velocity BC on BC nodes. It is easy to implement.

Unlumped Method #1 One approach to unlumping the boundary condition methods is to spread out the lumped method using the full mass matrix. This method would find forces from

$$\boldsymbol{f}^{(\#1)} \Delta t = \sum_{j} \frac{\tilde{\mathsf{m}}_{ij}}{m_j} \boldsymbol{f}_j^{(L)} = \tilde{\mathsf{m}} \mathsf{m}^{-1} \boldsymbol{f}^{(L)} \tag{2.46}$$

Remark: An algorithm for this method is:

1. Calculate change in grid velocities using lumped methods on nodes with boundary conditions. In other words, find

$$\Delta \boldsymbol{v}^{(BC)} = \sum_{b} \Delta \boldsymbol{v}^{(b)} \quad \text{where} \quad \Delta \boldsymbol{v}_i^{(b)} = \begin{cases} \left(v_i^{(b)} - \frac{\boldsymbol{p}_i^{+}}{m_i} \cdot \hat{\boldsymbol{n}}^{(b)} \right) \hat{\boldsymbol{n}}^{(b)} & i = j_b \\ 0 & i \neq j_b \end{cases}$$

2. Use full mass matrix to find:

$$\boldsymbol{p}^{(BC)} = \boldsymbol{f}^{(BC)} \Delta t = \tilde{\mathsf{m}} \Delta \boldsymbol{v}^{(BC)} \tag{2.47}$$

More explicitly:

$$\begin{aligned} \boldsymbol{p}_i^{(BC)} &= \sum_{b} \sum_{j} (\tilde{\mathsf{m}})_{ij} \boldsymbol{v}_j^{(b)} = \sum_{b} (\tilde{\mathsf{m}})_{i j_b} \left(v_{j_b}^{(b)} - \frac{\boldsymbol{p}_i^{+}}{m_i} \cdot \hat{\boldsymbol{n}}^{(b)} \right) \hat{\boldsymbol{n}}^{(b)} \\ &= \sum_{b} \sum_{p} M_p S_{pi} S_{p j_b} \left(v_{j_b}^{(b)} - \frac{\boldsymbol{p}_i^{+}}{m_i} \cdot \hat{\boldsymbol{n}}^{(b)} \right) \hat{\boldsymbol{n}}^{(b)} \end{aligned}$$

This results would take another extrapolation over particles, but particles with $S_{p j_b} = 0$ can be skipped (*i.e.*, those not in the domain of any boundary condition node.

3. Once boundary condition forces are found, add them to grid forces (or grid momentum in certain strain update methods). The reason to rely on a lumped $\Delta\boldsymbol{v}^{(b)}$ is to allow this calculation to be made before the XPIC calculations to convert momenta to velocities.

Remark: This approach does not reduce to lumped methods when using a lumped mass matrix unless $\tilde{\mathsf{m}}$ is replaced by m in above equations. Perhaps it should be replaced, in general, by $\tilde{\mathsf{m}}_k$. The next option does that is two places.

Unlumped Method #2 A second approach to unlumping the boundary conditions is use full mass matrix in the $\Delta\boldsymbol{v}$ calculation as follows:

1. Calculate change in grid velocities using full mass-matrix methods on nodes with boundary conditions. In other words, find

$$\Delta\boldsymbol{v}^{(BC)} = \sum_b \Delta\boldsymbol{v}^{(b)} \quad \text{where} \quad \Delta\boldsymbol{v}_i^{(b)} = \begin{cases} \left(v_i^{(b)} - (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_i \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)} & i = j_b \\ 0 & i \neq j_b \end{cases} \tag{2.48}$$

   where BC b is on j_b.

2. Use the approximate full mass matrix to find:

$$\boldsymbol{p}^{(BC)} = \boldsymbol{f}^{(BC)}\Delta t = \tilde{\mathsf{m}}_k \Delta\boldsymbol{v}^{(BC)}$$

   More explicitly:

$$\boldsymbol{p}_i^{(BC)} = \sum_b \sum_j (\tilde{\mathsf{m}}_k)_{ij} \Delta\boldsymbol{v}_j^{(b)} = \sum_b (\tilde{\mathsf{m}}_k)_{ij_b} \left(v_{j_b}^{(b)} - (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_{j_b} \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)}$$

3. If this boundary condition force could be evaluated (efficiently enough), the resulting nodal velocity would be

$$\begin{aligned}
\boldsymbol{v}^{+'} &= \tilde{\mathsf{m}}_k^{-1}\left(\boldsymbol{p}^+ + \boldsymbol{p}^{(BC)}\right) \\
\boldsymbol{v}_i^{+'} &= (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_i + \sum_b \sum_j (\tilde{\mathsf{m}}_k^{-1})_{ij} (\tilde{\mathsf{m}}_k)_{jj_b} \left(v_{j_b}^{(b)} - (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_{j_b} \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)} \\
&= (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_i + \sum_b \delta_{ij_b} \left(v_{j_b}^{(b)} - (\tilde{\mathsf{m}}_k^{-1}\boldsymbol{p}^+)_{j_b} \cdot \hat{\boldsymbol{n}}^{(b)}\right)\hat{\boldsymbol{n}}^{(b)} \\
&= \boldsymbol{v}_i^+ + \sum_b \Delta\boldsymbol{v}_i^{(b)}
\end{aligned}$$

Remark: leaving aside the question of whether or not it would be possible to find $\tilde{\mathsf{m}}_k$ used in the first two steps above, the final calculation seems to revert to the velocity method (including its limitations). An option to change might be to replace $\tilde{\mathsf{m}}_k$ with $\tilde{\mathsf{m}}$. The new result would get forces on other nodes, but would not exactly satisfy the boundary conditions. Or they could be combined to add the forces to other nodes, but enforce correct velocity on BC nodes?

2.5 Velocity Boundary Conditions - Revised

A problem noticed for FMPM grid BCs is that all or part of the BC methods used lumped mass matrix concepts, while velocity calculations use full mass matrix methods. The follow revisions seem to work much better.

1. Use the revised FMPM calculation in Section 1.4.3 based on $\Delta\boldsymbol{v}(\ell)$ method.
2. Use standard MPM methods to impose velocity BCs on updated lumped mass matrix velocity $\boldsymbol{v}^+(1)$.
3. After each calculation of $\Delta\boldsymbol{v}(\ell)$ impose zero velocity on boundary conditions nodes. The net result will be that $\boldsymbol{v}(k)$ will continue to satisfy boundary conditions.

The method seems to work. Can it be explained why? We start with

$$\boldsymbol{v}(1) = \boldsymbol{v}^{+'}(1)$$

as lumped-mass matrix grid velocities corrected by $\boldsymbol{f}_i^{(BC)}$ only on nodes and directions with velocity BCs using lumped mass matrix methods. Next, let $\Delta\boldsymbol{v}_\ell$ be changes in velocities needed to set grid BC velocities zero in $\Delta\boldsymbol{v}(\ell)$. Using lumped-mass methods on $\Delta\boldsymbol{v}(\ell)$, this velocity change implies an extra force on BC nodes is

$$\Delta\boldsymbol{v}_\ell = -\sum_b (\Delta\boldsymbol{v}(\ell)\cdot\hat{\boldsymbol{n}}^{(b)})\hat{\boldsymbol{n}}^{(b)} \implies \boldsymbol{f}_i^{(BC)}(\ell)\Delta t = -\sum_b m_i(\Delta\boldsymbol{v}(\ell)\cdot\hat{\boldsymbol{n}}^{(b)})\hat{\boldsymbol{n}}^{(b)}$$

The net result for FMPM(2) is

$$\boldsymbol{v}(2) = \boldsymbol{v}(1) + \Delta\boldsymbol{v}(2) + \Delta\boldsymbol{v}_2 = (\mathsf{I}+\mathsf{A})\boldsymbol{v}(1) + \Delta\boldsymbol{v}_2$$

Note that this result is identical to prior combined method because that set BC in $\boldsymbol{v}^+(1)$ and then set them again in $\boldsymbol{v}^{+'}(1) + \Delta\boldsymbol{v}(2)$ which is identical to setting to zero in $\Delta\boldsymbol{v}(2)$. Things change starting with FMPM(3) as

$$\begin{aligned}\boldsymbol{v}(3) &= (\mathsf{I}+\mathsf{A})\boldsymbol{v}(1) + \Delta\boldsymbol{v}_2 + \mathsf{A}(\mathsf{A}\boldsymbol{v}(1) + \Delta\boldsymbol{v}_2) + \Delta v_3 \\ &= (\mathsf{I}+\mathsf{A}+\mathsf{A}^2)\boldsymbol{v}(1) + (\mathsf{I}+\mathsf{A})\Delta\boldsymbol{v}_2 + \Delta v_3\end{aligned}$$

The generalization to any order is clear

$$\boldsymbol{v}(k) = \sum_{\ell=0}^{k-1}\mathsf{A}^\ell\boldsymbol{v}(1) + \sum_{\ell=2}^{k}\left(\sum_{j=0}^{k-\ell}\mathsf{A}^j\Delta\boldsymbol{v}_\ell\right)$$

This approach is the mechanical interpretation that velocity BCs can be satisfied only using reaction forces on nodes with velocity BCs. The net reaction forces on those nodes are:

$$\boldsymbol{f}_i^{(BC)}\Delta t = \sum_b m_i\left[v_i^{(b)} - \left(\boldsymbol{v}_i^+(1) + \sum_{\ell=2}^{k}\Delta\boldsymbol{v}(\ell)\right)\cdot\hat{\boldsymbol{n}}^{(b)}\right]\hat{\boldsymbol{n}}^{(b)}$$

The final velocities, however are change on nodes with velocity BCs and nodes without them. The one with velocity BCs are changed to the correct velocity. To see this result, expand final velocity and look at one BC using

$$\boldsymbol{v}(k) = \boldsymbol{v}^+(1) + \Delta v_1 + \sum_{\ell=2}^{k}\big(\Delta\boldsymbol{v}(\ell) + \Delta\boldsymbol{v}_\ell\big)$$

$$\begin{aligned}\boldsymbol{v}_i(k)\cdot\hat{\boldsymbol{n}}^{(b)} &= \boldsymbol{v}_i^+(1)\cdot\hat{\boldsymbol{n}}^{(b)} + v_i^{(b)} - \boldsymbol{v}_i^+(1)\cdot\hat{\boldsymbol{n}}^{(b)} + \sum_{\ell=2}^{k}\big(\Delta\boldsymbol{v}_i(\ell)\cdot\hat{\boldsymbol{n}}^{(b)} - \Delta\boldsymbol{v}_i(\ell)\cdot\hat{\boldsymbol{n}}^{(b)}\big) \\ &= v_i^{(b)}\end{aligned}$$

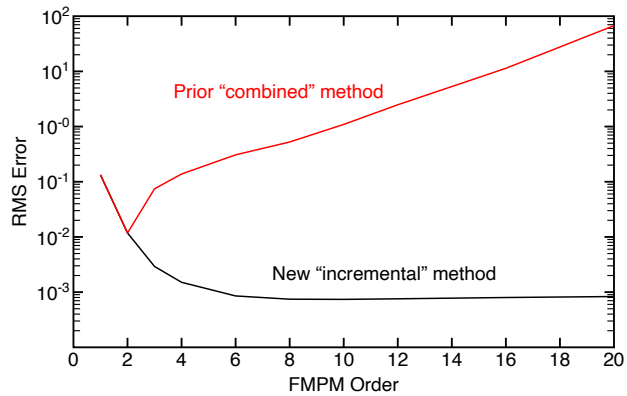

Figure 2.1: RMS error for uniaxial MMS using grid displacement BCs.

Or the final velocity satisfies velocity BCs exactly. Note that the old "Velocity" or "Combined" methods also satisfy velocity BCs, but keep the effects localized on the velocity BC nodes (fully localized in "Velocity" method). This new approach uses localized forces, but velocity effects spread to nearby nodes by full mass matrix effects.

A sample calculation in MMS for uniaxial loading shows this methods works (see Fig. 2.1). The results are identical for FMPM(1) (because that is a lumped mass matrix method) and for FMPM(2) (because combined method is setting zero velocity in the one increment). For higher order, the new method goes to very low error while old "combined" method get increasing errors. The accuracy plateaus for order 10.

Chapter 3

FMPM With Contact

3.1 Full Mass Matrix Equations

This section attempts to pose MPM contact using full mass matrices and vector equations. In multi-material mode, the extrapolations extrapolate each material to its own velocity field. We assume two materials α and β. The initial extrapolations will find $\boldsymbol{p}^\alpha$, $\boldsymbol{p}^\beta$, $\tilde{\mathsf{m}}^\alpha$, $\tilde{\mathsf{m}}^\beta$, m^α, and m^β all related to total momentum, mass matrix and lumped mass matrix by:

$$\boldsymbol{p} = \boldsymbol{p}^\alpha + \boldsymbol{p}^\beta, \qquad \tilde{\mathsf{m}} = \tilde{\mathsf{m}}^\alpha + \tilde{\mathsf{m}}^\beta, \qquad \text{and} \qquad \mathsf{m} = \mathsf{m}^\alpha + \mathsf{m}^\beta \tag{3.1}$$

Formally, materials α and β define separate spaces with different number of active nodes. Here the separate material matrices are expanded to the size of the all active nodes. The inverse of such matrix is the inverse found on with its local size and then expanded to full size by filling inactive rows and columns with zeros.

Now consider a set of velocities on the grid, found either from current or updated momenta on the grid by $\boldsymbol{v}^\alpha = (\tilde{\mathsf{m}}^\alpha)^{-1}\boldsymbol{p}^\alpha$ (and similar for β). The goal in contact mechanics is to change these velocities to $\boldsymbol{v}^\alpha + \Delta\boldsymbol{v}^\alpha$ and $\boldsymbol{v}^\beta + \Delta\boldsymbol{v}^\beta$ where they satisfy

$$\begin{aligned} \Delta\boldsymbol{v}_i^\beta - \Delta\boldsymbol{v}_i^\alpha &= k_i\hat{\boldsymbol{t}}_i - \left(\boldsymbol{v}_i^\beta - \boldsymbol{v}_i^\alpha\right) && \text{nodes in contact} \\ \Delta\boldsymbol{v}_i^\beta = \Delta\boldsymbol{v}_i^\alpha &= 0 && \text{nodes not in contact} \end{aligned}$$

This condition restricts motion on contact nodes to the tangential direction on node i ($\hat{\boldsymbol{t}}_i$) where k_i depends on contact law (*e.g.*, $k_i = 0$ for frictionless sliding). To conserve momentum, the velocity changes should be related to equal and opposite momenta added to nodes resulting in:

$$(\Delta\boldsymbol{v}^\alpha)_i = \left((\tilde{\mathsf{m}}^\alpha)^{-1}\Delta\boldsymbol{p}\right)_i \qquad \text{and} \qquad (\Delta\boldsymbol{v}^\beta)_i = -\left((\tilde{\mathsf{m}}^\beta)^{-1}\Delta\boldsymbol{p}\right)_i \tag{3.2}$$

The challenge is to find $\Delta\boldsymbol{p}$, which might span all nodes, to give localized nodal velocity changes listed above. By analogy to lumped mass matrix methods, we define $\tilde{\mathsf{m}}^{(red)} = \left((\tilde{\mathsf{m}}^\alpha)^{-1} + (\tilde{\mathsf{m}}^\beta)^{-1}\right)^{-1}$, which simplifies to:

$$-(\tilde{\mathsf{m}}^{(red)})^{-1}\Delta\boldsymbol{p} = \mathsf{k}\hat{\boldsymbol{t}} = \begin{cases} k_i\hat{\boldsymbol{t}}_i - \left((\tilde{\mathsf{m}}^\beta)^{-1}\boldsymbol{p}^\beta - (\tilde{\mathsf{m}}^\alpha)^{-1}\boldsymbol{p}^\alpha\right) & \text{nodes in contact} \\ 0 & \text{nodes not in contact} \end{cases} \tag{3.3}$$

where $\tilde{\mathsf{m}}^{(red)} = \left((\tilde{\mathsf{m}}^\alpha)^{-1} + (\tilde{\mathsf{m}}^\beta)^{-1}\right)^{-1}$, which simplifies to:

$$\tilde{\mathsf{m}}^{(red)} = \left((\tilde{\mathsf{m}}^\beta)^{-1}\tilde{\mathsf{m}}^\beta(\tilde{\mathsf{m}}^\alpha)^{-1} + (\tilde{\mathsf{m}}^\alpha)^{-1}\tilde{\mathsf{m}}^\alpha(\tilde{\mathsf{m}}^\beta)^{-1}\right)^{-1} = \tilde{\mathsf{m}}^{-1}\tilde{\mathsf{m}}^\alpha\tilde{\mathsf{m}}^\beta \tag{3.4}$$

This simplification used property that symmetric matrices commute. The solved $\Delta\boldsymbol{p}$ should satisfy:

$$\begin{aligned} \left((\tilde{\mathsf{m}}^{(red)})^{-1}\Delta\boldsymbol{p}\right)_i &= \left(\boldsymbol{v}_i^\beta - \boldsymbol{v}_i^\alpha\right) - k_i\hat{\boldsymbol{t}}_i && \text{nodes in contact} \\ \left((\tilde{\mathsf{m}}^\alpha)^{-1}\Delta\boldsymbol{p}\right)_i &= \left((\tilde{\mathsf{m}}^\beta)^{-1}\Delta\boldsymbol{p}\right)_i = 0 && \text{nodes not in contact} \end{aligned}$$

Finding the velocity difference directly from momenta (and exploiting symmetric mass matrices) gives:

$$\begin{aligned} \boldsymbol{v}^\beta - \boldsymbol{v}^\alpha &= (\tilde{\mathsf{m}}^\beta)^{-1}\boldsymbol{p}^\beta - (\tilde{\mathsf{m}}^\alpha)^{-1}\boldsymbol{p}^\alpha = (\tilde{\mathsf{m}}^\alpha)^{-1}(\tilde{\mathsf{m}}^\beta)^{-1}\left(\tilde{\mathsf{m}}^\alpha\boldsymbol{p}^\beta - \tilde{\mathsf{m}}^\beta\boldsymbol{p}^\alpha\right) \\ &= \left[(\tilde{\mathsf{m}}^{(red)})^{-1}\tilde{\mathsf{m}}^{-1}\left(\tilde{\mathsf{m}}^\alpha\boldsymbol{p}^\beta - \tilde{\mathsf{m}}^\beta\boldsymbol{p}^\alpha\right)\right] \end{aligned}$$

The nodes in contact change to

$$\left[(\tilde{\mathsf{m}}^{(red)})^{-1}\left(\Delta\boldsymbol{p} - \tilde{\mathsf{m}}^{-1}\left(\tilde{\mathsf{m}}^\alpha\boldsymbol{p}^\beta - \tilde{\mathsf{m}}^\beta\boldsymbol{p}^\alpha\right)\right)\right]_i = -k_i\hat{\boldsymbol{t}}_i$$

It does not seem feasible to solve for $\Delta\boldsymbol{p}$ that satisfies all these nodal velocity conditions. FMPM and XPIC likely need some approximate method (note that all full mass matrix is approximate too).

When using a lumped mass matrix, $\tilde{\mathsf{m}}_{ij}^{(red)} = m_i^\alpha m_i^\beta\,\delta_{ij}/m_i = m_i^{(red)}\delta_{ij}$. The momentum change on contact nodes become separate equations with solution

$$\Delta\boldsymbol{p}_i = \left(\frac{m_i^\alpha\boldsymbol{p}_i^\beta - m_i^\beta\boldsymbol{p}_i^\alpha}{m_i} - m_i^{(red)}\mathsf{k}_i\hat{\boldsymbol{t}}_i\right)\delta_{ic}\Delta\boldsymbol{p}_i = \left(\frac{m_i^\alpha\boldsymbol{p}_i^\beta - m_i^\beta\boldsymbol{p}_i^\alpha}{m_i} - m_i^{(red)}\mathsf{k}_i\hat{\boldsymbol{t}}_i\right)\delta_{ic} \tag{3.5}$$

where $\delta_{ic} = 1$ on nodes in contact and 0 on all other nodes These equations agree with MPM contact papers with general contact laws. In lunmped mass methods, the first term is

$$\Delta\boldsymbol{p}_i(0) = \frac{m_i^\alpha\boldsymbol{p}_i^\beta - m_i^\beta\boldsymbol{p}_i^\alpha}{m_i} \implies \Delta\boldsymbol{p}_i = \Delta\boldsymbol{p}_i(0) - m_i^{(red)}\mathsf{k}_i\hat{\boldsymbol{t}}_i$$

is the momentum change that must be applied to material α to change to the center of mass velocity. The tangential force needed to satisfy motions is

$$S_{slide}A_c = \boldsymbol{f}_i^{(c)} = \frac{\Delta\boldsymbol{p}_i\cdot\hat{\boldsymbol{t}}_i}{\Delta t} = \frac{\Delta\boldsymbol{p}_i(0)\cdot\hat{\boldsymbol{t}}_i - m_i^{(red)}\mathsf{k}_i}{\Delta t}$$

where S_{slide} is tangential traction in contact mechanics and A_c is the contact area. Solving for k_i gives

$$k_i = \frac{\Delta\boldsymbol{p}_i(0)\cdot\hat{\boldsymbol{t}}_i - S_{slide}A_c\Delta t}{m_i^{(red)}}$$

resulting in

$$\Delta\boldsymbol{p}_i = \Delta\boldsymbol{p}_i(0)\cdot\hat{\boldsymbol{n}}_i + (S_{slide}A_c\Delta t)\hat{\boldsymbol{t}}_i$$

The first term is normal contact force to avoid interpenetration and the second is tangential forces, which is zero for frictionless or non-zero for some contact law.

3.1.1 FMPM(k) Contact Options

It does not appear possible to solve exactly (*i.e.*, find Δp to satisfy nodal velocity changes) and many approximate options are available. Here are some ideas:

1. **No Change**: this method applies Δp_i by lumped mass method (and that is done after initial extrapolation, after momentum update, and after second extrapolation (if it is done). Follow this by no alteration of the FMPM loop (which is only done after momentum update when using `USL`-method, `USF` and `USL+` would add loops for other contact changes too). Coding is trivial because nothing done in FMPM loops.

2. **Incremental Corrections**: to previous method adds momentum to each node in contact an the subsequent FMPM loop will spread to momentum to nearby nodes. Some of that spread is good (effect of contact felt some distance from the node), but spread to other multimaterial modes could cause those node to violate contact conditions or to come into contact when they were not in contact before. A partial correction is to change $\Delta \boldsymbol{v}_i(\ell)$ on each node determined to be in contact be keep velocity different the same. In other words, change them to use a center of mass velocity or

$$\Delta \boldsymbol{v}_i^{\alpha *}(\ell) = \Delta \boldsymbol{v}_i^{\beta *}(\ell) = \frac{m_i^\alpha \Delta \boldsymbol{v}_i^\alpha(\ell) + m_i^\beta \Delta \boldsymbol{v}_i^\beta(\ell)}{m_i^\alpha + m_i^\beta}$$

   or to lumped-mass-matrix average of the two changes. Coding needs

   - When do contact, mark each contact node as whether or not is in in contact (for efficiency)
   - Each loop through revised FMPM loop make change in above equation. Maybe this can work directly with FMPM loop vectors and not need to a phase to copy to momentum vectors used by contact (needs new contact code for mulitmaterial and for cracks).
   - Do for `USL+`. If works extend to other methods. Will need to clear contact node flags for each new use.

3. **Post FMPM Loop**: this method delays contact calculations until after the FMPM loop. It uses full-mass-matrix velocities in contact calculations, but all contact effects on affect contact nodes only. The coding is

   - Skip full mass matrix contact code after momentum update. As a result, uncorrected, lumped-mass velocities will be input to the FMPM loop.
   - When FMPM loop is done, copy *e*ffective momenta using `mass*vk[VSTAR_VEC]`. Only need to do on contact nodes.
   - Do contact calculations and $\Delta \boldsymbol{p}_i / m_i$ to `vk[VSTAR_VEC]`.
   - Do for `USL+`. If works extend to other methods.

4. **No Change with Post-Loop Correction**. Start with "No Change" method but then repeat post-loop calculation by previous method.

3.1.2 Comparison of FMPM(k) Methods

All above methods are available is `OSParticulas`, revision 3924. The methods were evaluated by impact of two elastics disks (in 2D) approaching each other at 1% of their wave speed and then separating. The calculations looked at total energy, which should be conserved, after separation. The disks were 20 mm in diameter with $E = 1000$ MPa, $\nu = 0.33$, and $\rho = 1.5$ g/cm^3 and used the `Neohookean` material option. The calculations used CPDI shape functions, USL- update method, CFL factor 0.2, all in plane strain.

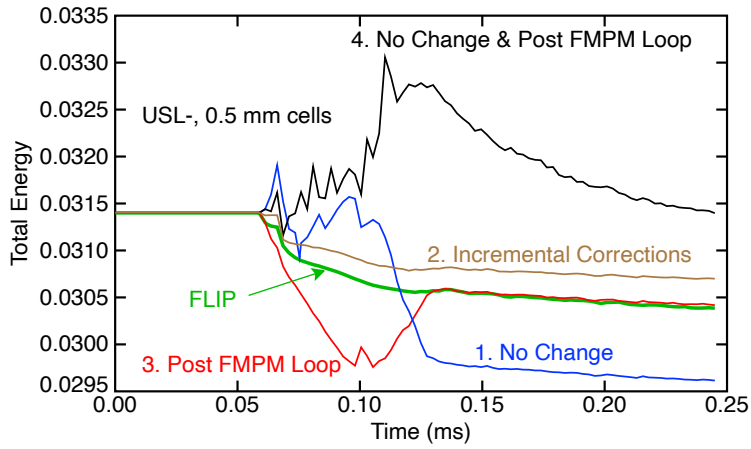

Figure 3.1: Total energy as a function of time for two elastic disks in impact by USL- with 0.5 mm cells (other details in text). The labels correspond to methods in the previous section and these are selected in revision 3924 by setting `MM_FMPM` to 0 to 3 corresponding to labels 1 to 4.

FLIP calculations were run first and varied resolution from 2 mm cells down to 0.125 mm cells. As expected, even low-dissipation FLIP dissipates energy at any finite resolution, but it does extrapolate toward zero dissipation at zero cell size (see below). Even when momentum is conserved in a method, no method exactly solves the momentum equation for this problem. Anyone proposing a method that conserves energy has probably derived a method that artificially adds energy to compensate for dissipated energy (an occasional misconception in the MPM literature). In fact, in this example, USL+ dissipates less energy, but that does not mean it is the preferred method. For comparison purposes. the following calculations mostly used 0.5 mm cells and USL-. At this resolution, FLIP dissipates 3.23% of the energy recorded at the end of the calculations. The calculations ran for 4× the impact time or about 0.245 ms (when disks start separated by 1 mm) and energy (sum of kinetic and work energy) was recorded about 0.186 mm after impact starts.

Figure 3.1 compares FLIP calculations (green line) to FMPM(8) calculations by the four methods listed in the previous section. FMPM(8) was selected (higher k than usual) to magnify any artifacts caused by FMPM. The "No Change" option is OK during impact, but ends up dissipating more energy. The "Incremental Corrections" has no artifacts during impact and dissipates less energy then FLIP (down to 2.23%). Methods 3 and 4 did not improve on method 2 (and caused more artifacts). By this test, method 2 or "Incremental Corrections" is the clear preference.

Note that "Incremental Corrections" is not much different than FLIP. This outcome is reasonable. This new FMPM(k) contact method is not implementing full mass matrix methods for contact, but rather allowing lumped-mass matrix contact methods to be compatible with XPIC or FMPM. If order of FMPM(k) is changed, it dissipated more energy then FLIP for $k < 4$ but less energy than FLIP for $k \geq 4$. Calculations done with FMPM(16) at 0.5 mm cell resolution dissipated 1.86%. Dissipation decreases as resolution increases (see below). Although difference between FLIP and FMPM for total energy dissipation are small, energy is not the only metric for quality of the results. The FMPM stress states are smoother than the FLIP stress states.

Figure 3.2A shows log-log spatial convergence of FLIP or FMPM(8) when using USL-. The results are similar and both are close to first order convergence (compared to dashed line). FMPM(8) is slightly better at all resolutions. Figure 3.2B shows convergence by all methods by both FLIP and FMPM(8). All methods are converging to zero. A first impression is that USL+, USF, USAVG+, and USAVG- (barely) improve on USL-, but that might not be the best impression. First, only USL- converges monotonically.

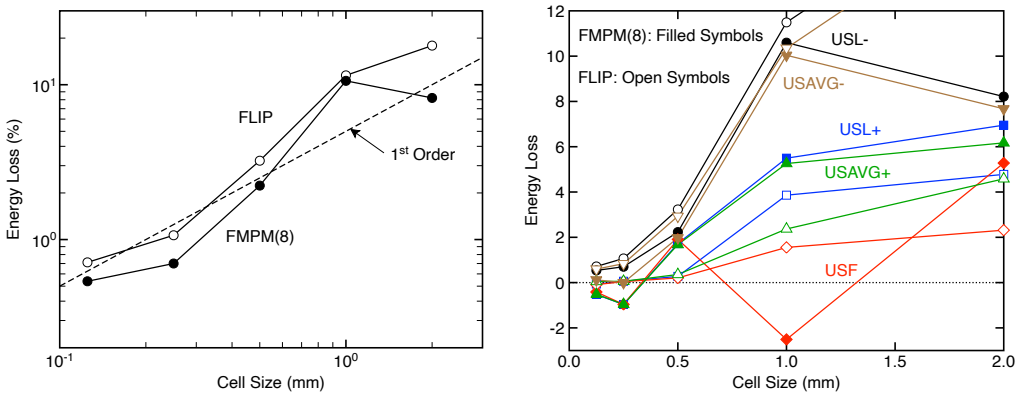

Figure 3.2: Caption

Other methods jump around and some start increasing energy before turning back to head to zero energy dissipation. Any method that causes energy to increase might not be safe method to use in general. It is better to have more dissipation in this problem provided convergence behaves as expected. Also note that FMPM(8) is slightly better than FLIP for USL- but is worse than FLIP for all other methods. Another issue is that plots of energy *vs.* time show more artifacts during the impact event in all methods except for USL- (for which virtually no artifacts are seen). Finally, USL- is the most efficient because it only needs one FMPM loop per time step (USF, USL+, and USAVG- need two and USAVG+ needs 3).

Artifacts in XPIC or FMPM were first noticed in shock calculations in paper on logistic regression. A method to correct was proposed in that paper for XPIC. Although it was proposed to use the same correction for FMPM, that fix did not work as well in other contact calculations (such as the two disk example above). These new four methods were repeated for the shock pressure wave. This example modeled two identical materials with or without a multimaterial interface; theoretically the results should be identical or the interface should be transparent as the pressure wave passes the interface. The "Incremental Corrections" option was again the best and in fact was identical to shock wave without the interface. The "No Changes" method has a large artifact at the interface. Methods 3 and 4 were unstable in this problem and failed soon after the pressure wave reached the interface.

Finally, the "Incremental Corrections" method using USL- is done only once per time step during particle updates and just after updating momentum on the grid. To traction contact forces, the velocity change should also increment force on the node using

$$\Delta \boldsymbol{f}_i^\alpha = \frac{m_i^\alpha \left(\Delta \boldsymbol{v}_i^{\alpha *}(\ell) - \Delta \boldsymbol{v}_i^\alpha(\ell)\right)}{\Delta t}$$

For non-rigid materials, this calculation is not needed by FMPM because nodal force is not used in subsequent particle updates. It is needed, however, when using XPIC. A comparable calculation is also needed to track contact forces on rigid materials. The above two-disk example above was repeated by changing one of the disks to be rigid with zero velocity. The contact force on the rigid material by FLIP or FMPM(8) are nearly identical when including this force calculation. If the force calculation is omitted, the calculations has some small artifacts.